31/7

Algorithms Analysis & Design

What's an $\underbrace{\text{algorithm}}$?

↪ Set of instructions to follow $\underbrace{\text{(Operational Definition)}}$

≡ Time : Angle by which seconds hand moves

Actual definition ⟶ Causes revolution

- What's a computer ?
- Seperate Hardware / Software
- Programming Language
- Computer
- What's a machine ?

Truth : Algorithm [Turing Algorithm]

→ Axioms (Assumptions) :

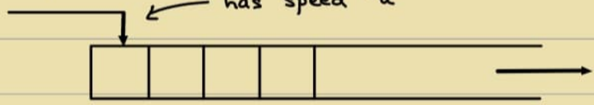
1) Information travels at a finite speed [Sp. Th. of relativity]

Loophole : RAM on Earth ⟹ malloc array in outer space

↪ (Printing array[100] or array[10^{100}]

Every $\underbrace{\text{memory}}$ is sequential ⟹ Memory ≡ Tape

has speed 'u'



↪ Only Sequentially accessible

Read - Write Memory

W.L.G One cell at a time (OR) can stay

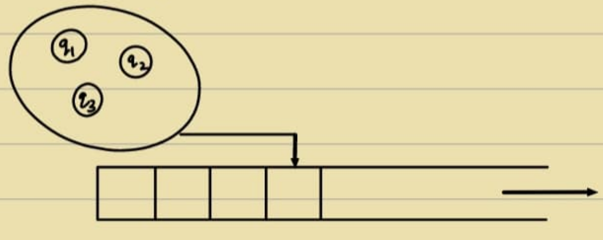
2) Finite volume in space can be used to reliably store/retrieve only finite amount of information. [Quantum Mech.]

e.g. Pendrive

Loophole : π

↪ Finite set of Tape Symbols (ε, T)

3) Only finitely many instructions can be 'packed' at a time



$q_i \in Q$: Finite set of states

Turing Machine

A T.M is a 7 tuple

$<Q, T, \delta, q_{start}, q_{accept}, q_{reject}>$ $\quad\quad \delta : T \times Q \longrightarrow T \times Q \times \{L, R\}$

7^{th} : Only occurs in Tape but not in Input

$\hookrightarrow \Sigma$: Finite set of input symbols , $\Sigma \subseteq T$

$b \in T, \; b \notin \Sigma$

$<Q, T, \Sigma, \delta, q_{start}, q_{accept}, q_{reject}>$

ALGORITHM : Simulatable in some TM (Church-Turing Thesis / Hypothesis)

$\underbrace{\text{Computer}}$: Algorithm

→ Realistic machines

→ Device which follows the axioms

VR : Anything that is real can also be virtual.

Single-structure programming Language ⟵ Turing Machine

Note : Turing Machine can be input to TM.

- $U_{TM}(<M>, x) = M(x)$

Universal Turing Machine : Universal General Purpose Software

x : Description of Circuit

M : Software

Software : Written on top of hardware

WHY :

$F_i = F_{i-1} + F_{i-2}$

$\left.\begin{array}{l} F(1) = 1 \\ F(0) = 0 \end{array}\right\}$ Recursion

Memoization } Iteration

$F_n = \frac{1}{\sqrt{5}}(\phi)^n - \frac{1}{\sqrt{5}}(\bar{\phi})^n$ } Formula $\quad\quad \left[\phi = \frac{1+\sqrt{5}}{2}\right]$

→ Eigenvalues [Hint to solve]

HOW :

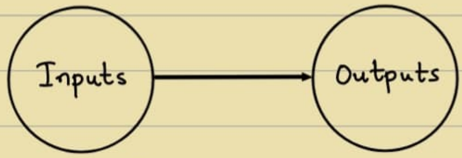
Algorithms book by DasGupta, Vaziram

→ Divide and Conquer
→ Greedy Algorithm
→ Dynammic Programming
→ Linear Programming
→ Complexity Theory
(Hardness)
→ Coping with Hard Problems
(Random, Approx, Quantum, etc)
→ Unsolvable problems

4/8

Q) WAP to ____
↳ Write A Program

→ What are computation problems ?



} → Decision Problems
→ Operational Definition

$\{0,1\}^*$ → Binary strings of finite length

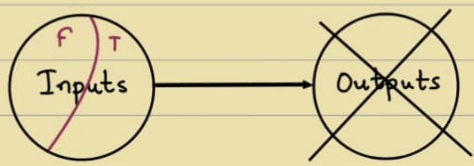
$\{0,1\}$ → WLG

Output → $\{0,1\}$

∵ If Output > 1 bit ⇒ Encode into multiple problems
i.e. if 01 → solution
0 : Solution of Problem 1 [2^1 place]
1 : Solution of Problem 2 [2^0 place]

No output ⇒ Not a computation problem (Acc. to course)

∴ $\underbrace{\text{Decision Problem}}$ → 2 - Partition of the input set
→ Language



Language $L \subseteq \{0,1\}^*$

<u>Note:</u> Decision Problem $\equiv$ Membership queries in Languages

Q) Given a graph, is it connected or not?

Graph $\longrightarrow$ Input

↳ Encode into $\{0,1\}^*$, i.e. Adjacency Matrix/List

[OR] $G \in \{ G' \mid G' \text{ is a connected Graph} \}$?

Q) Given a natural no., is it prime?

[OR] $n \in \{ i \mid i \text{ is prime} \}$

<u>Note:</u> If Answer is True/False $\Rightarrow$ Always decision problem

$\Downarrow$

Always computation problem

[e.g. Membership problem]

Q) WAP to print smallest prime no. $\equiv$ WAP to print smallest even no. ?

YES in formal Language

(NO in English)

[Can you formalize it or not ?] $\longleftarrow$ Bigger Question

↳ Computation problem or not ?

- e.g. LLMs : Speech to text C program
  - ↳ Creates problems rather than solving it lol when we don't define the problem
  - ↳ Poses non-CSE problems as CSE problems

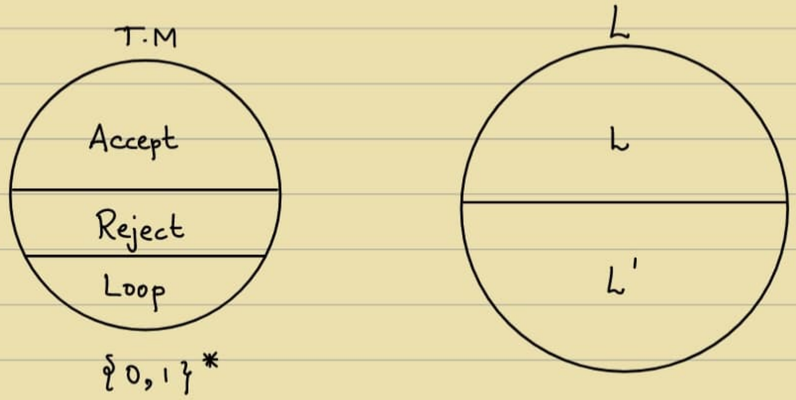
$*$ { Given a language $\longrightarrow$ We have a computation problem at hand

Given a T.M $\longrightarrow$ We have an answer / solution / program at hand }

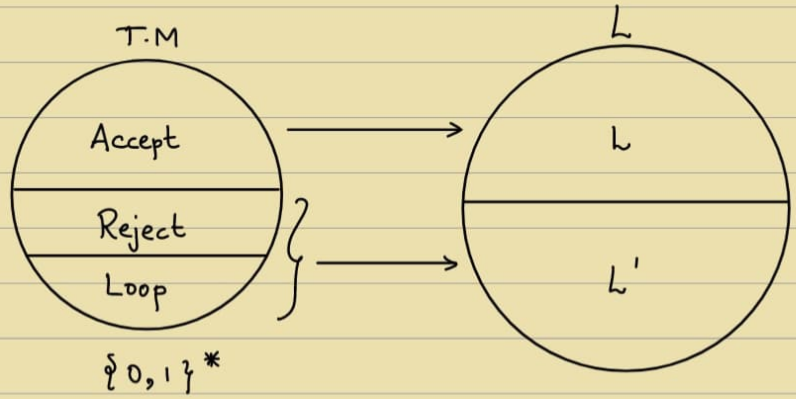
↳ What connects them ?

→ When do we say that a program solves a problem?

e.g. $n \in$ Primes ?



Q) When do we say a T.M solves a problem?



[Mapping]

<u>Def</u>

Def^n : A Language L is $\underbrace{\text{said}}$ to be recognized by T.M 'M' if
↳ Acc. to world

$\forall x \in L \rightarrow M(x)$ accepts

$\forall x \notin L \rightarrow M(x)$ does not accept

$L(M) = \{x \mid M(x) \text{ accepts}\}$

Def^n : A Language L is $\underbrace{\text{said}}$ to be decided by T.M 'M' if
↳ Acc. to world

$\forall x \in L \rightarrow M(x)$ accepts

$\forall x \notin L \rightarrow M(x)$ rejects

"This one, ChatGPT will also not know"

— AAD Prof, 2025

{ Active voice is preferred in Scientific Computer Literature } ☹

→ Towards Optimization :

- What are resources ?
- Quantifying their use
- Best Notions , etc

Computability $\underbrace{\text{Theory}}$ → What is a Theory ?

(Note : When something is an art , that means not everyone can do it

Science : Every man's art [No talent required]

Gifted Scientist ✗

Scientist ✓)

Theory gives you the answer to scientific problems.
when you can't perform scientific experiments.

$\underbrace{\text{Computability Theory}}$: Tells what can be solved , what can't

→ Does ∃ a C program ?

Functionality Theory :

TM → works in steps , ∴ Complexity can be measured

i.e. How much $\boxed{\text{memory}}$ it takes ?

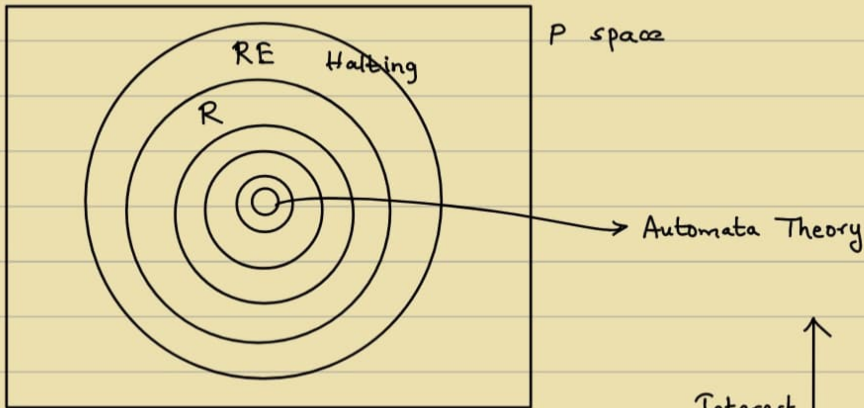
How much $\underbrace{\boxed{\text{time}}}$ it takes ?

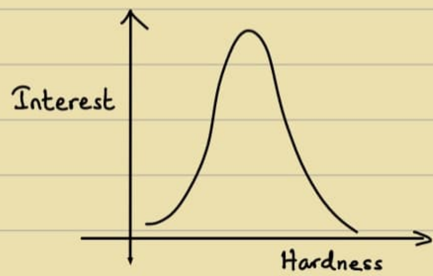
→ default

" Whenever there is a reality , there is a virtual reality "

<u>Theorem</u> : There are Languages that are unrecognizable

<u>Theorem</u> : There are Languages that are recognisable but undecidable.





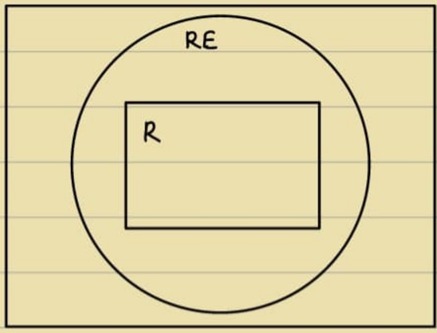
7/8

Def^n: The class of Recursively enumerable (RE) language is

$$RE = \{L \mid \exists \text{ T.M } 'M' \text{ that recognises } L\}$$

Def^n: The class of Recursive (R) language is

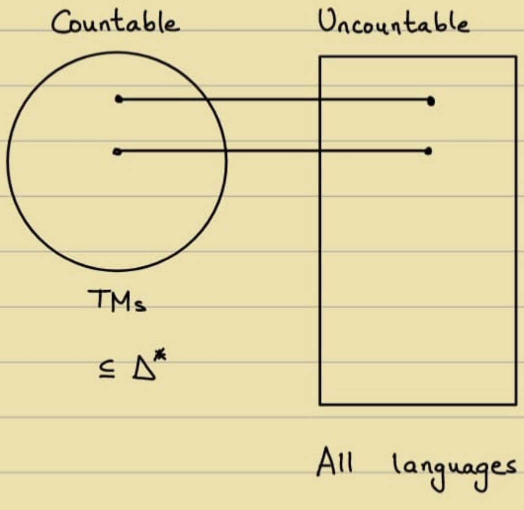
$$RE = \{L \mid \exists \text{ T.M } 'M' \text{ that decides } L\}$$



Q) Are there languages unrecognisable by TMs / Computers / C-Programs ?

Q) Are there undecidable languages ?

Proof:



Theorem: $\{0,1\}^*$ is countable

Proof: $f : \mathbb{N} \to \{0,1\}^*$

$\varepsilon, 0, 1, 00, 01, 10, 11, \underset{8}{\longleftrightarrow}, \underset{16}{\longleftrightarrow}$

$f(1) = \varepsilon$	$f(3) = 1$
$f(2) = 0$	$f(4) = 00$

Theorem: Set of all languages is uncountable.

Proof: A language $L \subseteq \{0,1\}^*$

Powerset of $\{0,1\}^* \rightarrow 2^{\{0,1\}^*}$

Every element in powerset can be uniquely described by a binary string.

$\therefore$ Bijection

e.g. $A = \{1, 2, 3\}$, $|A| = 3$

$P(A) = \{ \ldots, \underbrace{\{2,3\}}_{011}, \ldots \}$

1 is absent

2 & 3 are present

On the contrary, Suppose $2^{\{0,1\}^*}$ is countable

$f(1) = b_{11} b_{12} \ldots b_{1i} \ldots$

$f(2) = b_{21} b_{22} \ldots b_{2i} \ldots$

$\vdots$

$f(i) = b_{i1} b_{i2} \ldots b_{ii} \ldots$

Method of diagonalisation:

$L = b_{11} b_{22} b_{33} \ldots b_{ii} \ldots$

Can't be $f(1)$ $\because$ Differs atleast 1 location from $f(1)$

Can't be $f(2)$ $\because$ Differs atleast 1 location from $f(2)$

Can't be $f(i)$ $\because$ Differs atleast 1 location from $f(i)$

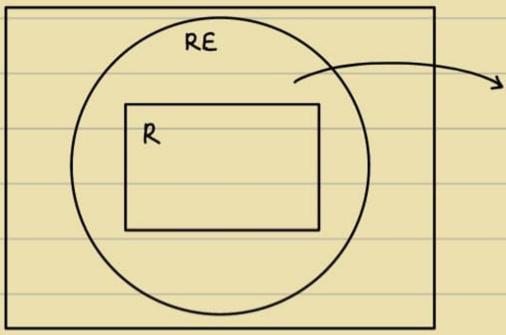
$\therefore$ Can't get bijection

$\therefore$ Can't be onto

$\therefore$ Uncountable

RE : Yes $\longrightarrow$ Yes

No $\longrightarrow$ No [OR] Unknown



Halting Problem, A_{TM}

- Given a TM 'M' and input 'x', does it accept x ?

$A_{TM} = \{ <M, x> \mid M \text{ accepts } x \}$

Yes $\longrightarrow$ Accept

No $\longrightarrow$ Reject [or] Run forever

$\therefore A_{TM} \in R.E$

$A_{TM} \notin R$

<u>Theorem</u>: A_{TM} is undecidable

<u>Proof</u>: Suppose A_{TM} is decidable

Let TM 'H' decide A_{TM}

$\forall (M, x) \in A_{TM}$, $H(M, x) = \text{Accept}$

$\forall (M, x) \notin A_{TM}$, $H(M, x) = \text{Reject}$ [H : Halting TM]

Consider a TM 'D'

On input $<M>$

→ Runs $H(M, <M>)$

→ If H accepts, REJECT

Else if H rejects, ACCEPT

$<M>$: Typecast program 'M' as strings

Execute $D(<D>)$

On input $<D>$,

→ Run $H(D, <D>)$

→ If H accepts, then REJECT

if D accepts $<D>$, then REJECT ← Which is impossible

→ If H rejects, then ACCEPT

If D does not accept $<D>$, then ACCEPT ← Which is illogical

$\therefore$ D does not exist $\Longrightarrow$ H does not exist

(OR) Proof by diagonalisation method

no. of TMs $\longrightarrow$ Countable

$\therefore$ Can be enumerable & orderable

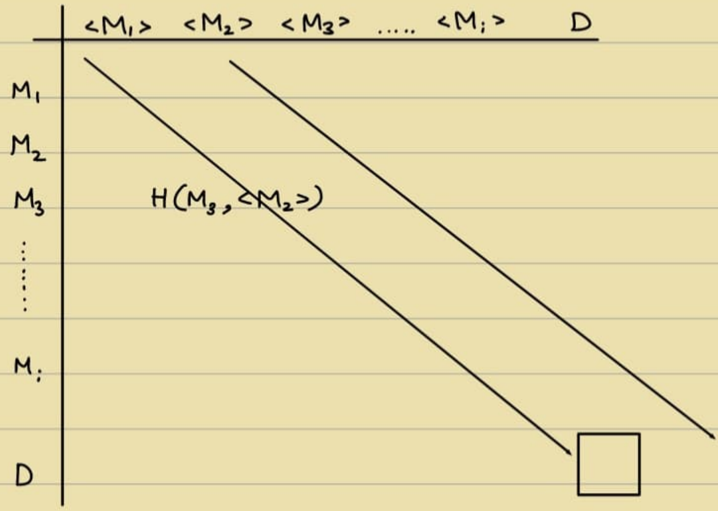


Table must have ACCEPT or REJECT.

But we get Blank

i.e. Whatever in diagonal, do opposite

$\therefore$ H cannot decide on all inputs

<u>Theorem</u>: If $L \in RE$, $\overline{L} \in RE$, then $L \in R$

<u>Proof</u>: $L \in RE$ $\exists$ TM 'M' S.T

$\forall x \in L \Rightarrow$ M accepts x

$\forall x \in \overline{L} \Rightarrow$ M does not accept x

$\overline{L} \in RE$ $\exists$ TM 'M'' S.T

$\forall x \in \overline{L} \Rightarrow$ M' accepts x

$\forall x \in L \Rightarrow$ M' does not accept x

Decider for L:

Just run M and M' in parallel

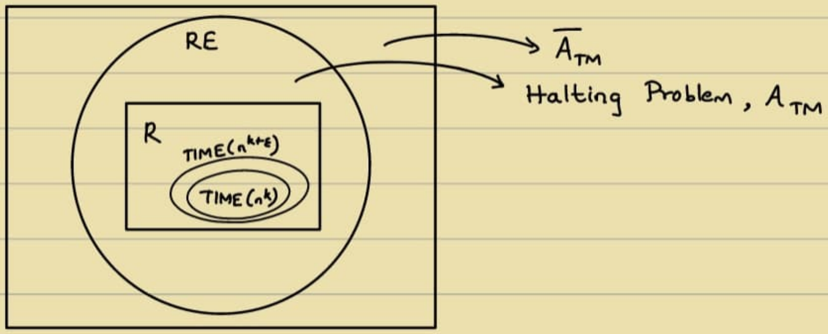
<u>Theorem</u>: $\overline{A_{TM}} \notin RE$

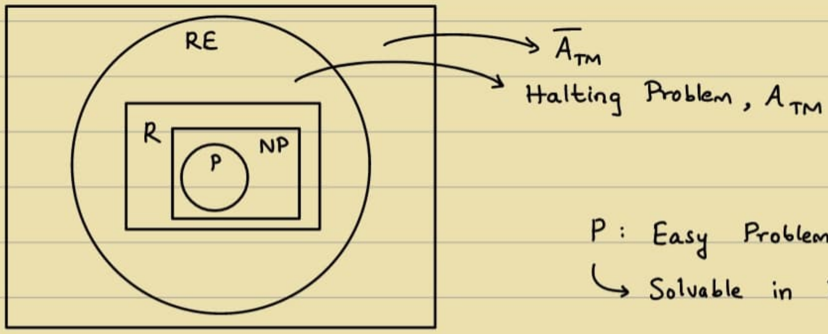
<u>Proof</u>: $\left.\begin{array}{l} A_{TM} \in RE \\ A_{TM} \notin R \end{array}\right\} \Rightarrow \therefore \overline{A_{TM}} \notin RE$

$\longrightarrow$ Take a class $O(n^k) \equiv TIME(n^k)$

Take another class $O(n^{k+\varepsilon}) \equiv TIME(n^{k+\varepsilon})$

For what value of ε, will they be diff. ?





P : Easy Problems
↳ Solvable in Polynomial time

- <u>Reduction</u> : (Technique)

  Problem A
  Problem B $\Big\}$ Given

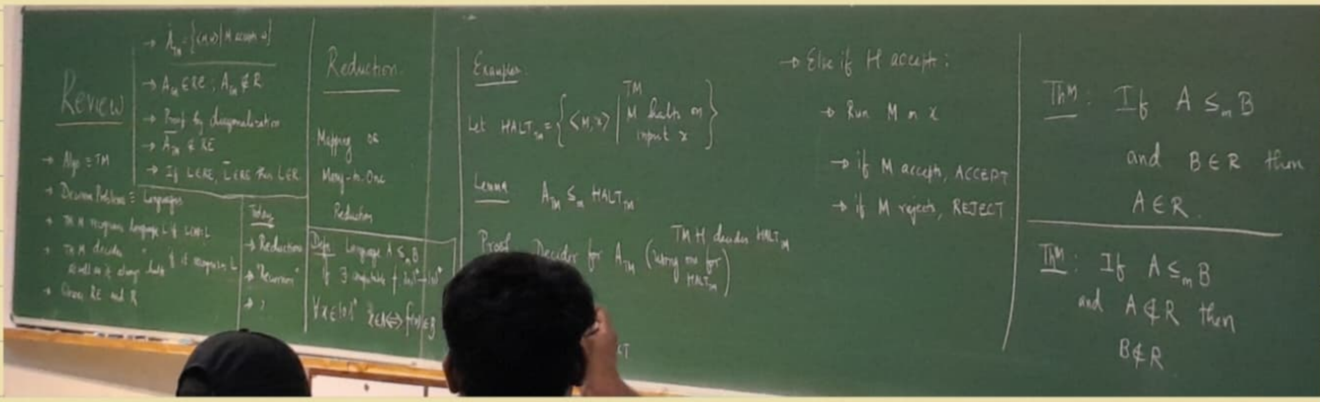
  Use Problem B as a subroutine to solve Problem A
  (Given there is a program for B.)

<u>Def^n</u> : A Language is mapping - reduced to L_2 if
$\exists$ computable function f, $f : \{0,1\}^* \longrightarrow \{0,1\}^*$ S.T.

$$\boxed{\forall x \in \{0,1\}^*, \quad x \in L_1 \iff f(x) \in L_2}$$

Denoted as $L_1 \leq_m L_2$

<u>11/8</u>



→ <u>Completeness Theory</u> :

Theorem : $Halt_{TM}$ is RE - Complete

Suppose $\forall A \in RE$, $A_M \leq Halt_{TM}$,

Then, $Halt_{TM}$ is RE - Complete

<u>Proof</u> :

Let TM 'M' recognise A &

H decides $Halt_{TM}$

Decider for A :

On input x, Run $H(M, x)$ if H reads REJECT

Else, Run M on x and do linking

→ $EQUAL_{TM} = \{<M_1, M_2> \mid L(M_1) = L(M_2)\}$

<u>Theorem</u> : $EQUAL_{TM}$ is unrecognisable

<u>Proof</u> : $\overline{A_{TM}}$ is unrecognizable ($\overline{A_{TM}} \notin RE$, known)

$M, x \in \overline{A_{TM}}$

Let E recognise $EQUAL_{TM}$

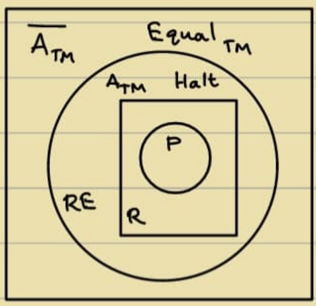
M_1 : On input w, REJECT

M_2 : On input w, if $x \neq w$, REJECT

if M accepts x, ACCEPT

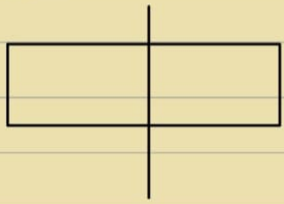
Else REJECT

if $E(M_1, M_2)$ accept $\Leftrightarrow$ M does not accept x



→ Divide and Conquer :

- Merge Sort



$T(1) = 1 \longrightarrow O(n)$

$T(n) = 2T(n/2) + O(n)$

$T(n) = O(n \log n)$

- Given $A = [\qquad]$ → n elements

  & $x \in A$

  What is rank(x) in A ?

  $O(n) \longrightarrow$ Easy

- Order Statistics

  Given $A = (\qquad)$ and index i

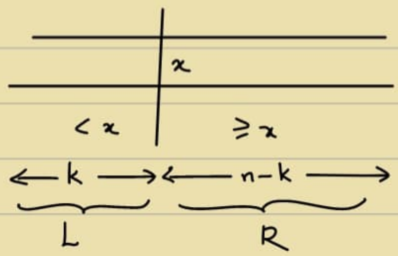
  Find the i^{th} ranked element in A

  $O(n\log n) \rightarrow$ Easy

  $O(n) \rightarrow$?

$S(A, i)$

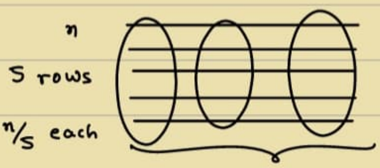
Pivot x



$$S(A,i) = \begin{cases} S(L,i), & \text{if } i \le k \\ S(R, i-k), & \text{if } i > k \end{cases}$$

$T(n) = O(n) + \max(T(k), T(n-k))$

Choose a 'good' x



$n/5$ medians

Let x be median of these $n/5$ medians

$S(n/5, n/10)$

$$T(n) = O(n) + T(n/5) + T(n/10)$$

$$T(n) = O(n)$$

$$T(n) = cn$$

$$\Rightarrow cn = O(n) + c(n/5) + c(n/10)$$

$$= O(n) + nc\underbrace{(1/5 + 1/10)}_{<1}$$

$$= (0.1)\, cn = O(n)$$

$n/3 + 2n/3 \longrightarrow$ Not < 1

$\therefore$ Won't work

Optimal group size ?

14/8

→ **Fast Fourrier Transform (FFT)** :

Input : Two polynomials of degree 'n-1' say $A(x)$ and $B(x)$

Output : Their product $C(x)$, i.e. $C(x) = A(x) \cdot B(x)$

$A[n] \quad B[n] \longrightarrow C[2n-1]$

$A[n]$ → Coefficient Array

$$A(x) = \sum_{i=0}^{n-1} a_i x^i \quad \Big| \quad B(x) = \sum_{i=0}^{n-1} b_i x^i$$

$$\underbrace{C(x) = \sum_{i=0}^{2n-2} c_i x^i}$$

→ c_i : 'Direct Formula'

$$\sum_{j=0}^{i} a_j b_{i-j}$$

→ $O(n^2)$ algorithm

Can we do better ? i.e. $O(n\log n)$? FFT

Algorithm 'style'

1. Select $(2n-1)$ or more points $x_1, x_2, x_3, \ldots\ldots, x_n$ $[O(n)]$
2. Evaluate $A(x_i)$ and $B(x_i)$ $[O(n^2)]$
3. Multiply $A(x_i) \cdot B(x_i) = C(x_i)$ $[O(n)]$
4. Interpolate $C(x_i)$'s to obtain $C(x)$ $[O(n^3)]$

Task : ② $\rightarrow O(n^2)$ to $O(n\log n)$

$$A(x) = A_e(x^2) + x A_o(x^2)$$

e.g. $A(x) = 5x^4 - 3x^3 + 7x^2 + 2x + 1$

$$A_e(x) = 5x^2 + 7x^2 + 1$$

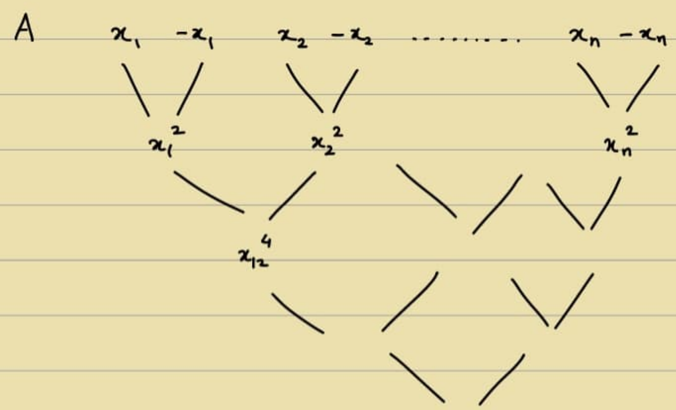
$$A_o(x) = -3x + 2$$

$$A(x_i) = A_e(x_i^2) + x_i \cdot A_o(x_i^2)$$

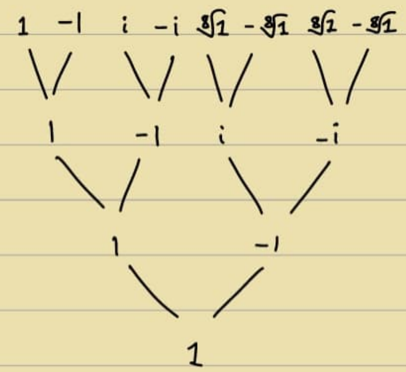
$$A(-x_i) = A_e(x_i^2) - x_i \cdot A_o(x_i^2)$$

$$T(n) = 2T(n/2) + O(n)$$

$$A_e(x_i^2) = A_{ee}(x_i^2) + x^2 \cdot A_{eo}(x_i^2)$$



n^{th} roots of unity



8^{th} roots of unity

$m \geq 2n-1$, $e^{i \cdot \frac{2k\pi}{n}}$

Goal : To evaluate a polynomial 'A' on the m^{th} roots of unity
$1, \omega, \omega^2, \ldots\ldots, \omega^{m-1}$

Input : A[ ]

Output : $(A[1], A[\omega], A[\omega^2], \ldots\ldots, A[\omega^{m-1}])$

$$A = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_m \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & 1 & 1 & \cdots \\ 1 & \omega & \omega^2 & \omega^4 & \cdots \\ 1 & \omega^2 & \omega^4 & \cdots & \\ 1 & \omega^4 & \vdots & \cdots & \end{bmatrix}_{m\times m} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{m-1} \end{bmatrix} = \begin{bmatrix} A(1) \\ A(\omega) \\ A(\omega^2) \\ \vdots \\ A(\omega^{n-1}) \end{bmatrix}$$

1, i entry

Discrete Fourrier Transform

[Interpolation]

```
FFT(A, ω):
    if (ω == 1)
        return A(1);
    else
        Ae ← [A(0), A(ω), ......]
        Ao ← [A(1), A(i), ......]

        E[] ← FFT(Ae, ω²)
        O[] ← FFT(Ao, ω²)

        for i=0 to m-1
            A[i] ← E[i/2] + ω^i · O[i/2]

        for i =0 to m/2
            A[i] ← E[i] + ω^i · O[i]
            A[i+m/2] ← E[i] + ω^i · O[i]
```

$$A(x) = A_e(x^2) + x \cdot A_o(x^2)$$
$$A(\omega^i) = E(\omega^{2i}) + \omega^i \cdot O(\omega^{2i})$$
$$A(-\omega^i) = E(\omega^{2i}) - \omega^i \cdot O(\omega^{2i})$$

$\therefore O(m\log n)$

multiplication

Algorithm 'style'

1. Select $m \geq (2n-1)$ or more points, namely m^{th} roots of unity
   $1, \omega, \omega^2, \omega^3, \ldots\ldots, \omega^{m-1}$ $\left(\omega = e^{i \cdot \frac{2k\pi}{m}}\right)$ (m: exact power of 2)
2. Evaluate $A(x_i)$ and $B(x_i)$ $[O(n \cdot \log n)]$
   FFT(A, ω) & FFT(B, ω)
3. Multiply $A(x_i) \cdot B(x_i) = C(x_i)$ $[O(n)]$
4. Interpolate $C(x_i)$'s to obtain $c(x)$ $[O(n^3)]$

$$C(x) = A(x) \cdot B(x)$$
$$C(1) = A(1) \cdot B(1)$$
$$C(\omega) = A(\omega) \cdot B(\omega)$$
$$C(\omega^2) = A(\omega^2) \cdot B(\omega^2)$$

$$\begin{bmatrix} \\ & M & \\ \\ \end{bmatrix}_{m\times m} \cdot \begin{pmatrix} c_0 \\ c_1 \\ \vdots \\ c_{m-1} \end{pmatrix} = \begin{pmatrix} c(1) \\ c(\omega) \\ c(\omega^2) \\ \vdots \\ c(\omega^{m-1}) \end{pmatrix}$$

$$M_{ij} = \omega^{ij}$$
$$\omega = e^{i \cdot \frac{2\pi}{m}}$$

$$M(\omega) = (i,j)^{th} \text{ entry}, \ \omega^{ij}$$

$$\begin{bmatrix} c_0 \\ c_2 \\ \vdots \\ c_{m-1} \end{bmatrix} = M^{-1} \begin{bmatrix} c(1) \\ c(\omega) \\ \vdots \\ c(\omega^{m-1}) \end{bmatrix}$$

$$\underset{\omega^{ij}}{\begin{bmatrix} & & \\ & & \\ & & \end{bmatrix}_{m\times m}} \underset{\omega^{-ij}}{\begin{bmatrix} & & \\ & & \\ & & \end{bmatrix}} = \begin{bmatrix} m & 0 & \cdots & 0 \\ 0 & m & 0 \cdots & 0 \\ 0 & 0 & m & \cdots \\ \vdots & \vdots & \vdots & \vdots \\ 0 & \cdots & \cdots & 0 \end{bmatrix}$$

$$M^{-1} = \frac{1}{m} M(\omega^{-1}) \qquad \frac{1}{m} FFT(m\omega^{-1})$$

18/8, 21/8, 25/8

Absent - Not well

28/8

→ Greedy Strategy :

- Minimum Spanning Tree — Krushkal, Prims, etc
- Activity Selection
- Knapsack Problem
- Huffman coding
- Djikstra's shortest path algorithm

→ Matroid Theory :

Def^n: A matroid is a pair $\langle S, I \rangle$, where

(a) S is a finite non-empty set

(b) I is a family of subsets of S. (Set of independent sets)

① Heredetiary Property : If $A \subseteq S$ belongs to I (if $A \in I$),
Then, $\forall B \subseteq A$, $B \in I$

② Exchange Property : If $A \in I$ and $B \in I$, and $|B| > |A|$,

Then, $\exists x \in B/A$ S.T. $A \cup \{x\} \in I$

Q) Given a matroid $<S, I>$ and given non-negative integer weight to element of S.

Find the maximum weight element of I. (A subset of S)

$$w(A) = \sum_{x \in A} w(x)$$

$x \in S$

$w(x)$ is the weight of x

- Universal Greedy Algo for weighted Matroids :

(1) Sort the elements of S in non-increasing order of weights.

(2) $A \leftarrow \phi$

(3) In that order, let x be the root element

if $A \cup \{x\} \in I$, then $A \leftarrow A \cup \{x\}$

- Proof of correctness / optimality :

Greedy-choice Property : The element $x \in S$ with the maximum weight belongs to some optimum solution.

Proof :

Let $A \in I$ be an optimum solution

if $x \in A$, done

Suppose $x \notin A$, $\{x\} \in I$

Let $A = \{a_1, a_2, \ldots\ldots, a_k\}$

$\exists B = (A \cup \{x\}) - \{a_i\}$, $B \in I$

$w(B) = w(x) + w(A) - w(a_i) \geq w(A)$

- Optimal sub-structure property :

M contracts to M'

$<S, I>$ $<S', I'>$

$A = A' \cup \{x\}$

$S' = S - \{x\}$

I' = Sets of I with x erased

$= \{B \mid B \cup \{x\} \in I\}$

All sets in I that originally contained x

- Graph Matroid :

Given a edge-weighted graph $G = (V, E)$

Let $S_G = E$

$I_G = \{ E' \subseteq E \mid E' \text{ does not contain a cycle} \}$

$M_G = \langle S_G, I_G \rangle$ is a matroid.

I_G is hereditary.

I_G has exchange property

$|E_1| > |E_2|$

$E_2 \to$ Forest with $|V| - |E_2|$ Trees

$E_1 \to$ Forest with exactly $|V| - |E_1|$ Trees

E_1 has an edge (u, v) S.T. u and v are in different trees in E_2.

$E_2 \in I$

$E_2 \cup \{(u,v)\} \in I_G$

$\therefore$ Weighted Matroid

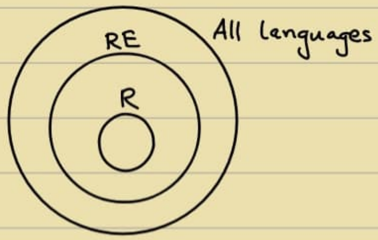
Quiz - 1

→ Knapsack Problem ✓
Greedy ✓
Activity Selection ✓
Krushkal's Algorithm ✓
Prim's Algorithm ✓
Cut Property ✓
Minimum Spanning Tree ✓

Strassen's Technique ✓ $O(n^{2.87})$
Binary Search ✓
Sorting - Counting Sort, Radix Sort
Merge Sort ✓
Karatsuba's Algorithm
Djikstra's Algorithm ✓
Master's Theorem ✓
Fast Fourier Transform ✓

$Equal_{TM}$
$A_{TM} = \{<M, x>, M \text{ accepts } x\}$
$Halting_{TM}$
Completeness Theory
Reduction

→ Recursive : Decidable ← Turing Machine ⇒ Accept if in L, Reject otherwise
Recursive Enumerable : Recognisable ← TM ⇒ Accept if in L, Halt/Loop otherwise
↳ Partially / Semi - Decidable



Case - I : $\log_b a > k$
$\therefore T(n) = \Theta(n^{\log_b a})$

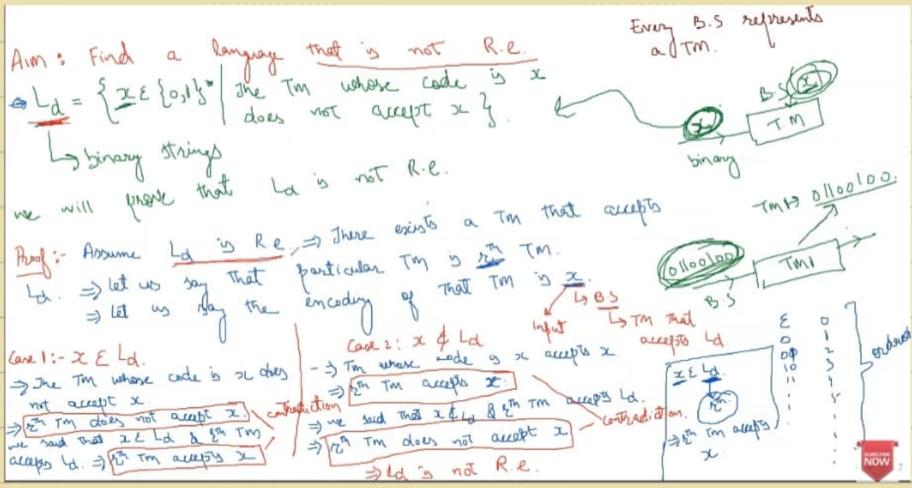
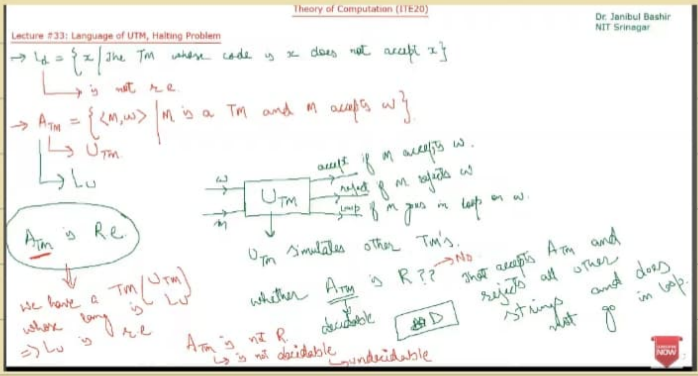
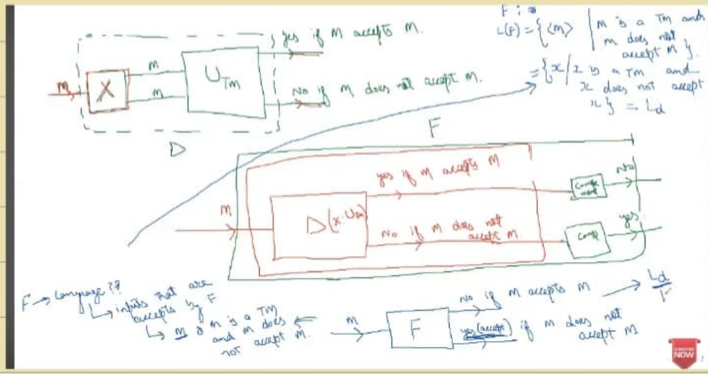
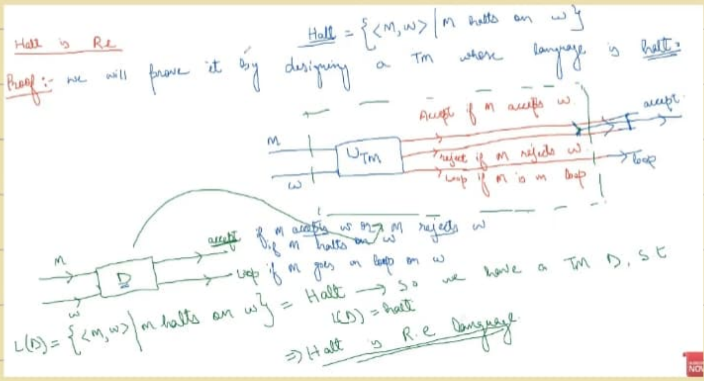
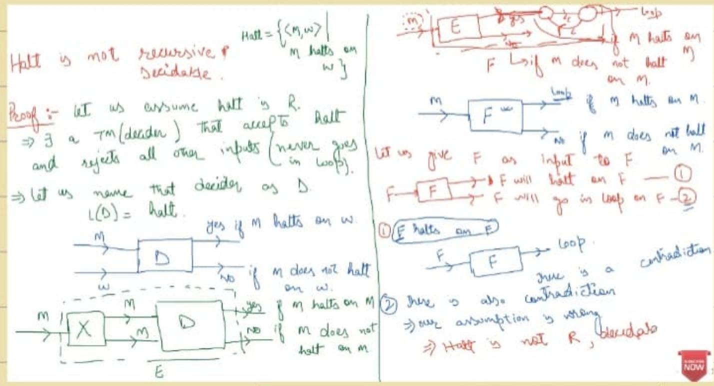
Case - II : $\log_b a = k$
(i) If $p > -1$:
$\therefore T(n) = \Theta(n^k \log^{p+1} n)$
(ii) If $p = -1$:
$\therefore T(n) = \Theta(n^k \log(\log n))$
(iii) If $p < -1$:
$\therefore T(n) = \Theta(n^k)$

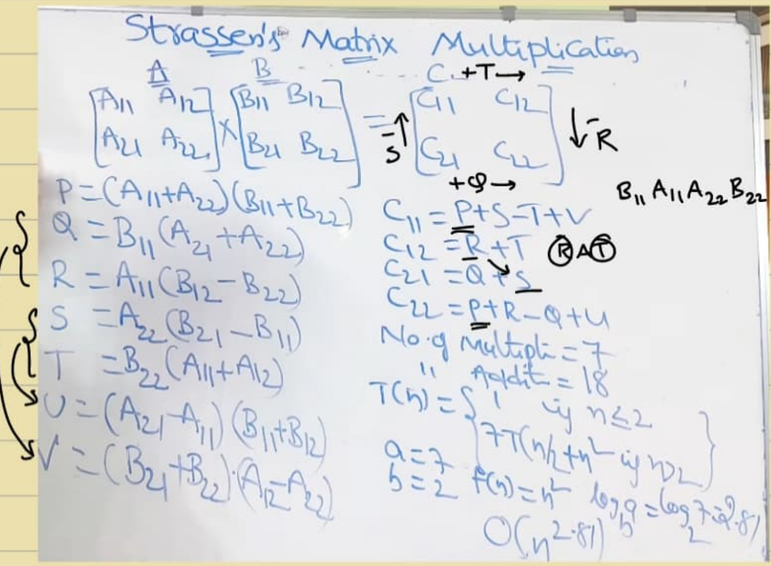
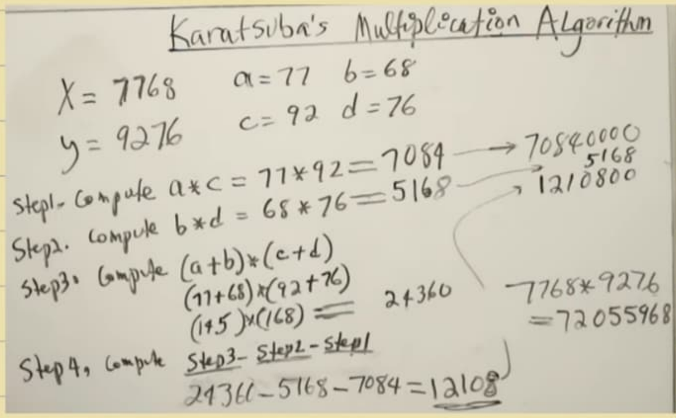
Case - III : $\log_b a < k$
(i) If $p >= 0$:
$\therefore T(n) = \Theta(n^k \log^p n)$
(ii) If $p < 0$:
$\therefore T(n) = \Theta(n^k)$

$T(n) = aT(n/b) + f(n)$

Conditions :
$a >= 1$,
$b > 1$,
$k >= 0$,
$p \in \mathbb{R}$

[where $f(n) = \Theta(n^k \log^p n)$]


Aim: Find a language that is not R.e.
Every B.S represents a TM.
Ld = { x ∈ {0,1}* | The TM whose code is x does not accept x }.
binary strings
we will prove that Ld is not R.e.
Proof:- Assume Ld is R.e. ⇒ There exists a TM that accepts Ld. ⇒ let us say that particular TM is x^th TM.
⇒ let us say the encoding of that TM is x.
Case 1:- x ∈ Ld.
Case 2: x ∉ Ld
contradiction
⇒ Ld is not R.e.

Theory of Computation (ITE20)
Lecture #33: Language of UTM, Halting Problem
Dr. Janibul Bashir
NIT Srinagar
Ld = { x | The TM whose code is x does not accept x }
is not r.e.
A_TM = { <M,w> | M is a TM and M accepts w }
U_TM
Lu
A_TM is R.e.
U_TM simulates other TM's.
A_TM is not R.
is not decidable
undecidable

yes if M accepts M.
No if M does not accept M.
U_TM
D
F
Ld

Halt is Re
Halt = { <M,w> | M halts on w }
Proof:- we will prove it by designing a TM whose language is halt.
U_TM
accept
loop
L(D) = { <M,w> | M halts on w } = Halt
⇒ Halt is R.e. language

Halt is not recursive / Decidable
Halt = { <M,w> | M halts on w }
Proof:- let us assume halt is R.
⇒ let us name that decider as D.
L(D) = halt.
yes if M halts on w.
No if M does not halt on w.
contradiction
⇒ Halt is not R, decidable

Let us assume that we can:
H(P, I)
Halt
Not Halt
This allows us to write another Program:
C(X)
if (H(X, X) == Halt)
Loop Forever;
else
Return;
If we run 'C' on itself:
C(C)
H (C,C) == Halt
Not Halt
H (C,C) == Not Halt
Halt

Strassen's Matrix Multiplication
P = (A11 + A22)(B11 + B22)
Q = B11(A21 + A22)
R = A11(B12 − B22)
S = A22(B21 − B11)
T = B22(A11 + A12)
U = (A21 − A11)(B11 + B12)
V = (B21 + B22)(A12 − A22)
C11 = P + S − T + V
C12 = R + T
C21 = Q + S
C22 = P + R − Q + U
No. of Multipli = 7
Additi = 18
O(n^2.81)

Karatsuba's Multiplication Algorithm
X = 7768 a = 77 b = 68
y = 9276 c = 92 d = 76
Step1- Compute a*c = 77*92 = 7084
Step2. Compute b*d = 68*76 = 5168
Step3. Compute (a+b)*(c+d)
Step4. Compute Step3 − Step2 − Step1
24360 − 5168 − 7084 = 12108
7768*9276 = 72055968

```
def FFT(P):
    # P - [p_0, p_1, ..., p_{n-1}] coeff representation
    n = len(P) # n is a power of 2
    if n == 1:
        return P
    ω = e^(2πi/n)
    P_e, P_o = [p_0, p_2, ..., p_{n-2}], [p_1, p_3, ..., p_{n-1}]
    y_e, y_o = FFT(P_e), FFT(P_o)
    y = [0] * n
    for j in range(n/2):
        y[j] = y_e[j] + ω^j y_o[j]
        y[j + n/2] = y_e[j] − ω^j y_o[j]
    return y
```

$\text{FFT}\quad P(x) : [p_0, p_1, \ldots, p_{n-1}]\quad \omega = e^{\frac{2\pi i}{n}} : [\omega^0, \omega^1, \ldots, \omega^{n-1}]$

$\boxed{n = 1 \Rightarrow P(1)}$

$\text{FFT}\quad P_e(x^2) : [p_0, p_2, \ldots, p_{n-2}]\quad [\omega^0, \omega^2, \ldots, \omega^{n-2}]$

$y_e = [P_e(\omega^0), P_e(\omega^2), \ldots, P_e(\omega^{n-2})]$

$\text{FFT}\quad P_o(x^2) : [p_1, p_3, \ldots, p_{n-1}]\quad [\omega^0, \omega^2, \ldots, \omega^{n-2}]$

$y_o = [P_o(\omega^0), P_o(\omega^2), \ldots, P_o(\omega^{n-2})]$

$P(\omega^j) = y_e[j] + \omega^j y_o[j])$

$P(\omega^{j+n/2}) = y_e[j] - \omega^j y_o[j])$

$j \in \{0, 1, \ldots (n/2 - 1)\}$

$y = [P(\omega^0), P(\omega^1), \ldots, P(\omega^{n-1})]$

IFFT(<values>) ⇔ FFT(<values>) with $\omega = \frac{1}{n} e^{\frac{-2\pi i}{n}}$

```
def FFT(P):
    # P - [p_0, p_1, ..., p_{n-1}] coeff rep
    n = len(P) # n is a power of 2
    if n == 1:
        return P
    ω = e^(2πi/n)
    P_e, P_o = P[::2], P[1::2]
    y_e, y_o = FFT(P_e), FFT(P_o)
    y = [0] * n
    for j in range(n/2):
        y[j] = y_e[j] + ω^j y_o[j]
        y[j + n/2] = y_e[j] − ω^j y_o[j]
    return y
```

```
def IFFT(P):
    # P - [P(ω^0), P(ω^1), ..., P(ω^{n-1})] value rep
    n = len(P) # n is a power of 2
    if n == 1:
        return P
    ω = (1/n) * e^(−2πi/n)
    P_e, P_o = P[::2], P[1::2]
    y_e, y_o = IFFT(P_e), IFFT(P_o)
    y = [0] * n
    for j in range(n/2):
        y[j] = y_e[j] + ω^j y_o[j]
        y[j + n/2] = y_e[j] − ω^j y_o[j]
    return y
```



Harith

Yerragolam

– Sami, 2025

8/9

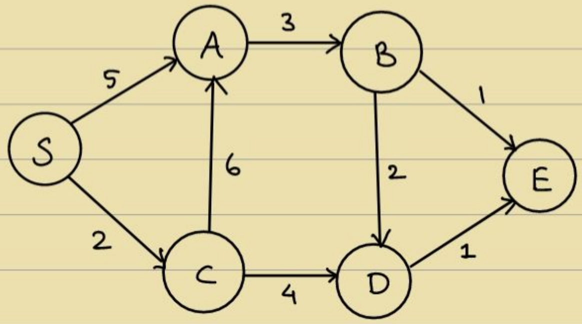
→ Dynamic Programming :

- A bag of subproblems S.T. it forms a DAG of 'smallest' to the 'largest' in Topological Sorted order.
- Problem : Shortest Path in DAGs

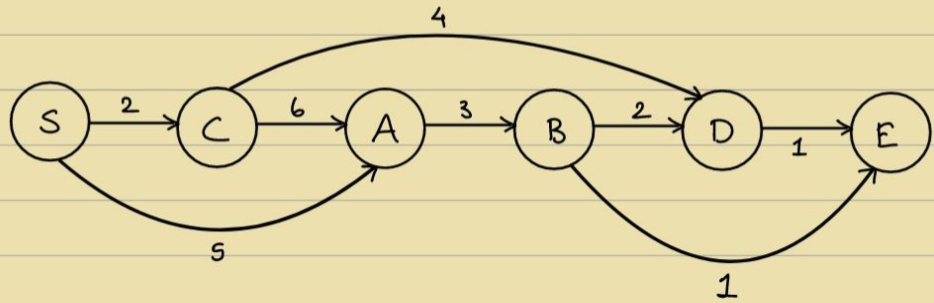
  Input : Weighted Directed Acyclic Graph
  Source S and Destination D

  Output : Shortest path from S and D.

Example :



Topological sort :



$$P(S, D) = \min \begin{cases} 2 + P(S, B) \\ 4 + P(S, C) \end{cases}$$

Termination condition : $P(S, S) = 0$

0	2	5	8	6	X
S→S	S→C	S→A	S→B	S→D	S→E

in-neighbours

$$P(S, i) = \min_{(u,i) \in E} \left(P(S, u) + D(u, i) \right)$$

$S \rightarrow A$: $\min(5, 8)$

$S \rightarrow D$: $\min(8, 10)$

- Longest path in DAG:

$$P(S, i) = \max_{(u,i) \in E} \left(P(S,u) + D(u,i) \right)$$

Topological sorting: $O(V+E)$

0	2	8	11	13	14
$S \to S$	$S \to C$	$S \to A$	$S \to B$	$S \to D$	$S \to E$

If efficient algorithm for finding longest path for general graphs, then $P = NP$

- Longest Increasing Subsequence:
  - Input: Array of numbers
  - Output: LIS

Every substring is a $\underbrace{\text{subsequence}}_{}$, but not vice-versa. $\to$ may not be continuous

e.g. $a_1, a_2, a_3, \ldots\ldots a_n$

$$\left.\begin{array}{l} a_{i_1} < a_{i_2} < a_{i_3} < \ldots\ldots < a_{i_n} \\ \text{where } i_1 < i_2 < i_3 < \ldots\ldots < i_n \end{array}\right\} \longrightarrow \text{Sequence}$$

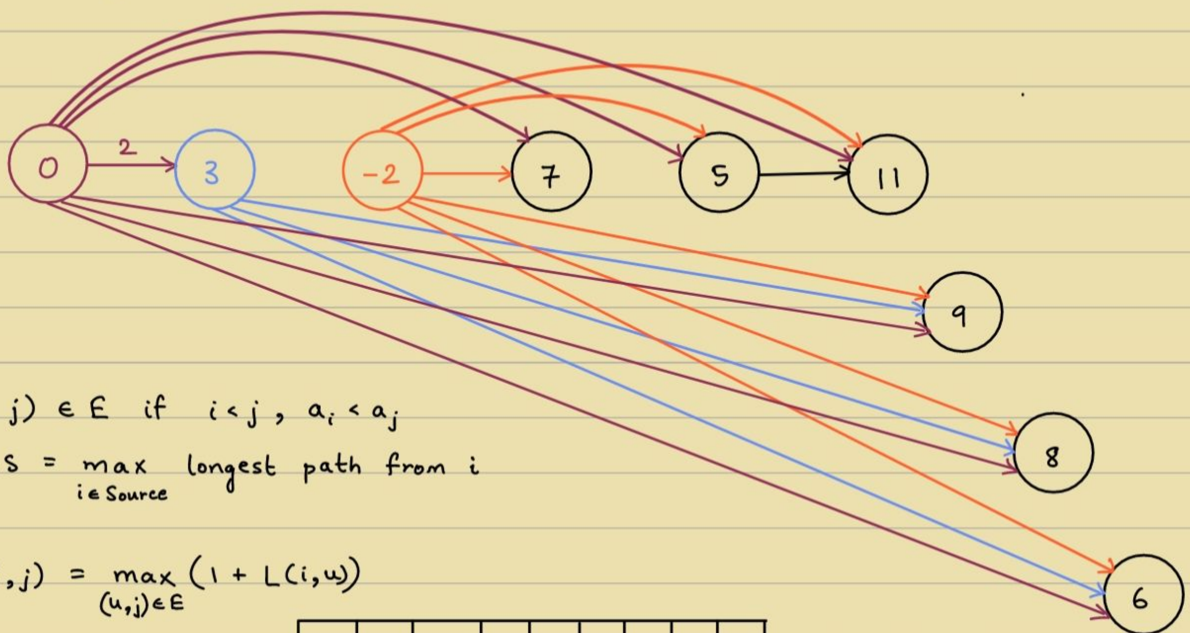
e.g. 0, 3, −2, 7, 5, 11, 9, 8, 6

Sol: 0, 3, 7, 11 $\longrightarrow$ length: 4

Viterbi's Algo, CYK Algo : Dynamic Programming

Viterbi's Algo $\downarrow$ Signal Processing

CYK Algo $\downarrow$ ToC



$(i, j) \in E$ if $i < j$, $a_i < a_j$

$$LIS = \max_{i \in \text{Source}} \text{longest path from } i$$

$$L(i,j) = \max_{(u,j) \in E} \left(1 + L(i,u) \right)$$

0	1	$-\infty$	2	2	3	3	3	3
0	3	−2	7	5	11	9	8	6

$\Rightarrow$ 0, 3, 7, 11

- <u>Edit Distance</u> : (Levenshtein distance)

$O(|E| \cdot k)$

$\underbrace{O(n^2)}_{}$ $\longrightarrow$ $O(n\log n)$?

Worst case scenario

e.g. EARTH $\longrightarrow$ HEART

① Insert H

HEARTH

② Delete H

HEART

Operations : Insert , Delete , Overwrite

e.g. SUNNY $\longrightarrow$ SNOWY

① Overwrite U $\longrightarrow$ N

② Overwrite N $\longrightarrow$ O

③ Overwrite N $\longrightarrow$ W

WAP that input 2 strings and finds the edit distance b/w them.

11/9

<u>Review</u> :

Dynamic Programming

e.g. ① Shortest / Longest Path in DAG

② Longest Increasing Subsequence

- Common Traits of D.P :
  - Bag / DAG of Subproblems
  - Subproblems are usually some part of the Input.
  - Recursive relation

- Edit Distance :

$x = x_1 x_2 x_3 \ldots\ldots x_n$

$y = y_1 y_2 y_3 \ldots\ldots\ldots y_m$

Add as many blanks to x and y S.T. they are aligned.

Cost of Allignment : No. of mismatches

EARTH $\Longrightarrow$ _EARTH

HEART $\quad$ HEART_

Subproblems :

(prefix) 1^{st} i characters of x

(prefix) 1^{st} j characters of y

$$\underbrace{E(i,j)} \longrightarrow \text{Goal} : E(n,m)$$

$$\left.\begin{array}{l} x_1 x_2 \ldots\ldots x_i \\ y_1 y_2 \ldots\ldots y_j \end{array}\right\} \text{Three cases :}$$

(a) $\left.\begin{array}{c} x_i \\ - \end{array}\right\} \Rightarrow E(i,j) = E(i-1,j) + 1$

(b) $\left.\begin{array}{c} - \\ y_j \end{array}\right\} \Rightarrow E(i,j) = E(i,j-1) + 1$

(c) $\left.\begin{array}{c} x_i \\ y_j \end{array}\right\} \Rightarrow E(i,j) = \begin{cases} E(i-1,j-1) & x_i = y_j \\ E(i-1,j-1) + 1 & x_i \neq y_j \end{cases}$

$$\therefore \; E(i,j) = \min \begin{pmatrix} 1 + E(i-1,j) \\ 1 + E(i,j-1) \\ \text{diff}(x_i,y_j) + E(i-1,j-1) \end{pmatrix}$$

$$\text{diff}(x_i,y_j) = \begin{cases} 0, & x_i = y_j \\ 1, & x_i \neq y_j \end{cases}$$

Termination condition :

$E(0,j) = j$

$E(i,0) = i$

		x_1	x_2	x_3	....	x_n
	0	1	2	3	....	n
y_1	1					
y_2	2					
y_3	3					
⋮	⋮					
y_m	m					$E(n,m)$

Fill it row-wise

to not get stuck anywhere.

e.g.

		H	E	A	R	T
	0	1	2	3	4	5
E	1	1	1 →	2 →	3 →	4
A	2	2	2	1	2	3
R	3	3	3	2	1	2
T	4	4	4	3	2	1
H	5	4	5	4	3	2

e.g. E X P O N E N T I A L

P O L Y N O M I A L

		E	X	P	O	N	E	N	T	I	A	L
	0	1	2	3	4	5	6	7	8	9	10	11
P	1	1	1	2	3	4	5	6	7	8	9	10
O	2	2	2	3	2	3	4	5	6	7	8	9
L	3	3	3	3	3	3	4	5	6	7	8	9
Y	4	4	4	4	4	4	4	5	6	7	8	9
N	5	5	5	5	5	4	5	4	5	6	7	8
O	6	6	6	6	5	5	5	5	5	6	7	8
M	7	7	7	7	6	6	6	6	$\boxed{6}$	6	7	8
I	8	8	8	8	7	7	7	7	7	6	7	8
A	9	9	9	9	9	8	8	8	8	7	8	9
L	10	10	10	10	9	9	9	9	9	8	7	$\boxed{6}$

Note : IAL is not required ∵ both strings end with 'IAL'

Answer : 6

Working :

$dp[m+1][n+1]$

where $dp[i][j]$: min. edits to convert first i chars of EXPONENTIAL to first j chars of POLYNOMIAL

$dp[0][j] = j$

$dp[i][0] = i$

for $i>0$, $j>0$:

if $s_1[i-1] == s_2[j-1]$:

$dp[i][j] = dp[i-1][j-1]$

else :

$$dp[i][j] = 1 + \min(\underbrace{dp[i-1][j]}_{\text{Delete}}, \underbrace{dp[i][j-1]}_{\text{Insert}}, \underbrace{dp[i-1][j-1]}_{\text{Replace}})$$

Result : $dp[i][10] = \underline{6}$

15/9

→ **Review** :

Dynamic Programming

Ex 1 : Shortest / Longest Path in DAG

Ex 2 : Longest Increasing Subsequence

Ex 3 : Edit distance

Ex. 4 : Chain Matrix Multiplication

→ **General Technique** :

- Creating subproblems using either prefix or some part of input.
- Relate the subproblems from easier to less easy in topological sorted order.

Input : Chain of Matrices

$$M^{(1)}_{P_0 \times P_1},\ M^{(2)}_{P_1 \times P_2},\ M^{(3)}_{P_2 \times P_3},\ \cdots\cdots,\ M^{(n)}_{P_{n-1} \times P_n}$$

Output : $M_{P_0 \times P_n} = \prod_{i=1}^{n} M_{P_{i-1} \times P_i}$

Cost : no. of elementary multiplications

Note : Notation :

$$M_{p,q} \times M_{q \times r} \longrightarrow pqr \text{ multiplications}$$

Example : $M^{(1)}_{100 \times 10} \times M^{(2)}_{10 \times 1000} \times M^{(3)}_{1000 \times 1} \times M^{(4)}_{1 \times 100}$

Total multiplications : $10^6 + 10^5 + 10^7 = 11100000$

$$\left(\left(M^{(1)} \times M^{(2)} \right) \times \left(M^{(3)} \times M^{(4)} \right) \right)$$

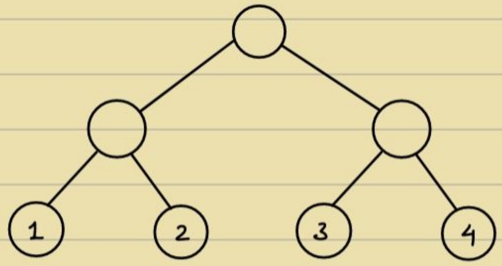
But, $\left[M^{(1)} \times \left(M^{(2)} \times M^{(3)} \right) \right] \times M^{(4)}$

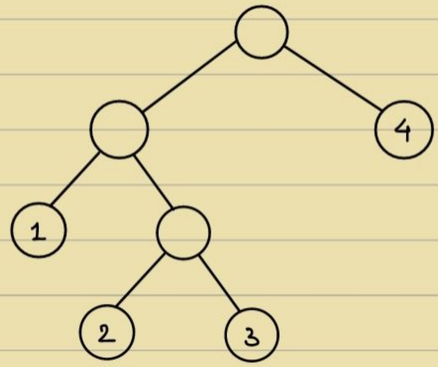
$$10^3 + 10^4 + 10^4 = 21000$$

Q) How many different ways are there to paranthesize a chain of n matrices ?

$$\frac{1}{n+1} \binom{2n}{n}$$

[Catalan numbers]





$C(i, j)$

Goal : To find $C(1, n)$

$C(i, i+1) = P_{i-1} P_i P_{i+1}$ ⟵ Easy ones

$\left[M^{(i)}_{P_{i-1} \times P_i} \right]$

Now,

Only P_i's are given

$C(i,j) = C(1, k) + C(k+1, j) + P_i P_j P_k$

(if last step was given)

Suppose $(M^{(i)} \times M^{(k)}) \times (M^{(k+1)} \times M^{(j)})$

Termination condition : $C(i, i) = 0$

$$C(i,j) = \min_{i<k<j} \left\{ C(1, k) + C(k+1, j) + P_{i-1} P_j P_k \right\}$$

	1	2	3	4
1	0	10^6	11000	21000
2		0	10^4	12000
3			0	10^5
4				0

Goal

Not necessary

$$C(1,3) = \min \begin{cases} C(1,2) + C(3,3) + P_0 P_2 P_3 \\ C(1,1) + C(2,3) + P_0 P_1 P_3 \end{cases}$$

$$C(1,3) = \min \begin{cases} C(1,3) + C(3,4) + P_1 P_3 P_4 \\ C(2,2) + C(2,4) + P_1 P_2 P_4 \end{cases} = \min \begin{cases} 11000 + 1000 \\ 10^5 + \dots \end{cases}$$

$$C(1,3) = \min \begin{cases} C(1,1) + C(2,4) + P_0 P_1 P_4 \\ C(1,2) + C(3,4) + P_0 P_2 P_4 \\ C(1,3) + C(4,4) + P_0 P_3 P_4 \end{cases}$$

18/9

→ **Review :**

D·P :

① Paths in DAG

② LIS

③ Edit distance

④ Chain Matrix Multiplication

→ Ex. Shortest Reliable Paths :

Input : Graph (Edge weighted) , s (start) , t (end) , integer k

Output : The shortest path from s to t in G using $\leq k$ edges

- Subproblems :

$\forall x \in V$, $\forall i < k$

Let $dist(u, i)$ be the length be the length of the shortest path from s to u , using i hops

- Main problem :

$\min\limits_{i \leq k} dist(t, i)$

Recurrence Relation :

$$dist(u, i) = \min_{(v,u) \in E} \left(dist(v, i-1) + \omega(v, u) \right)$$

Easiest Subproblem :

$dist(s, 0) = 0$ (Termination condition)

$dist(s, >0) = \infty$ (OR) $dist(\underset{\neq s}{u}, 0) = \infty$

→ Ex. All Pair shortest Path : $O(n^3)$

Input : G

Output : $\forall u, v$, length of shortest path from u to v.

- Subproblems :

Let dist (u, v, k) be the length of the length of the shortest path from u to v , using minimalistic nodes among 1, 2, ..., k

- Main Problem :

$k = |V|$

Recurrance relation :

$$dist(u, v, k) = \min\left(dist(u, v, k-1),\ dist(u, k, k-1) + dist(k, v, k-1) \right)$$

$$dist(u, v, 0) = \begin{cases} \omega(u, v) , & \text{if } (u,v) \in E \\ \infty , & \text{otherwise} \end{cases}$$

Normal $\rightarrow O(|E| \cdot \log(|V|))$

With fibonacii heap $\rightarrow O(|E| + |V| \log(|V|))$

→ Review for midsem:

① Dynamic Programming
- Shortest / longest Path in DAG
- Longest Increasing Subsequence
- Edit Distance
- All Pair shortest Path
- Chain Matrix Multiplication
- Shortest Reliable Path
- Knapsack
- Independent set in Tree
- Polygon Triangulation
- Other questions in Prob. set

② Greedy Algorithm
- Huffman codes
- Djikstra's Algo
- Knapsack (0-1 & Fractional)
- Activity Selection
- Minimum / Maximum Spanning Tree (Prims Algo, Krushkal's Algo)
- Matroid Theorem
- Other questions in Prob. set

③ Divide and Conquer
- Merge Sort
- Integer Multiplication
- Matrix Multiplication
- Polynomial Multiplication
- Other questions in Prob. set

④ Computability Theory
- Church - Turing Thesis
- Model of Computation (TM)
- Diagonalization
- (a) Recognizability
  (b) Decidability
- Mapping Reduction

Mid Sem

- Binary Search

```
T(n) ——— Algorithm RBinSearch(l,h,key)
{
     if(l==h)
     {
1 ———   if(A[l]==key)
            return l;
        else return 0;
     }
     else
     {
1 ———   mid=(l+h)/2;
1 ———   if(key==A[mid])
            return mid;
1 ———   if(key<A[mid])
T(n/2) ——— return RBinSearch(l,mid-1,key);
        else
            return RBinSearch(mid+1,h,key);
     }
}
```

$$T(n) = \begin{cases} 1 & , n = 1 \\ T(n/2) & , n \geq 1 \end{cases}$$

- Merge Sort :

```
Algorithm MergeSort(l, h)
{
   if(l<h)
   {
      mid=(l+h)/2;
      MergeSort(l,mid);
      MergeSort(mid+1,h);
      Merge(l,mid,h);
   }
}
```

```
Algorithm Merge(A,B,m,n)
{
   i=1; j=1; k=1;
   while(i<=m && j<=n)
   {
      if(A[i]<B[j])
         C[k++]=A[i++];
      else
         C[k++]=B[j++];
   }
   for( ; i<=m; i++)
      C[k++]=A[i];
   for( ; j<=n; j++)
      C[k++]=B[j];
}
```

$$T(n) = \begin{cases} 1 & , n = 1 \\ 2T(n/2) + n & , n > 1 \end{cases}$$

- <u>MST</u> :

no. of MSTs = ${}^{|E|}C_{|V|-1}$ − no. of cycles

Prims → Initially, take the min. cost edge
(Tree) Then, take the connected min. cost edge.

Krushkals → Always take the min. cost edge. → $O(|V|\cdot|E|)$
($\neg$Cycle) Don't consider loop edges. Min heap → $O(|V|\cdot\log|V|)$

Dijkstra's → $O(|V|\cdot|V|)$

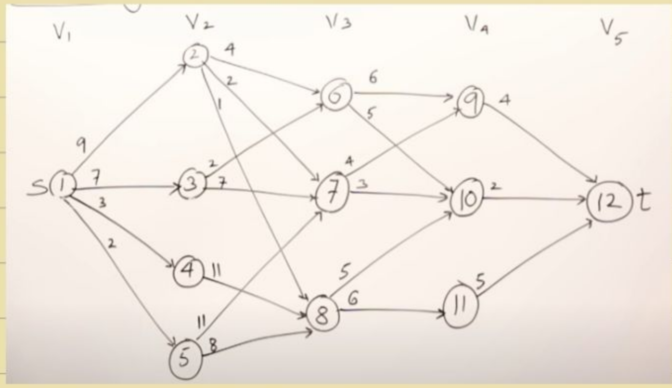
- **Dynamic Programming :**

  - Multi-Stage Graph : (Shortest Path in DAG)

stage ↓ vertex no ↓

$$Cost(i,j) = \min_{\substack{\langle j,l\rangle \in E \\ l \in V_{i+1}}} \{ c(j,l) + Cost(i+1,l) \}$$

V	1	2	3	4	5	6	7	8	9	10	11	12
Cost	16	7	9	18	15	7	5	7	4	2	5	0
d	2/3	7	6	8	8	10	10	10	12	12	12	12



for longest Path → $\underbrace{\text{Multiply by } -1}_{\text{costs}}$

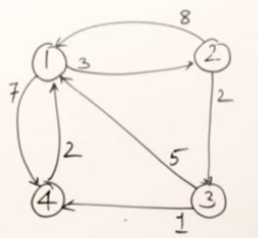
& Multiply resut by −1 to get correct cost.

  - All pair shortest Path : (Floyd – Warshall)

All Pairs Shortest Path

$$A^1 = \begin{array}{c|cccc} & 1 & 2 & 3 & 4 \\ \hline 1 & 0 & 3 & \infty & 7 \\ 2 & 8 & 0 & 2 & 15 \\ 3 & 5 & 8 & 0 & 1 \\ 4 & 2 & 5 & \infty & 0 \end{array} \qquad A^2 = \begin{array}{c|cccc} & 1 & 2 & 3 & 4 \\ \hline 1 & 0 & 3 & 5 & 7 \\ 2 & 8 & 0 & 2 & 15 \\ 3 & 5 & 8 & 0 & 1 \\ 4 & 2 & 5 & 7 & 0 \end{array}$$

$$A^3 = \begin{array}{c|cccc} & 1 & 2 & 3 & 4 \\ \hline 1 & 0 & 3 & 5 & 6 \\ 2 & 7 & 0 & 2 & 3 \\ 3 & 5 & 8 & 0 & 1 \\ 4 & 2 & 5 & 7 & 0 \end{array} \qquad A^4 = \begin{array}{c|cccc} & 1 & 2 & 3 & 4 \\ \hline 1 & 0 & 3 & 5 & 6 \\ 2 & 5 & 0 & 2 & 3 \\ 3 & 3 & 6 & 0 & 1 \\ 4 & 2 & 5 & 7 & 0 \end{array}$$

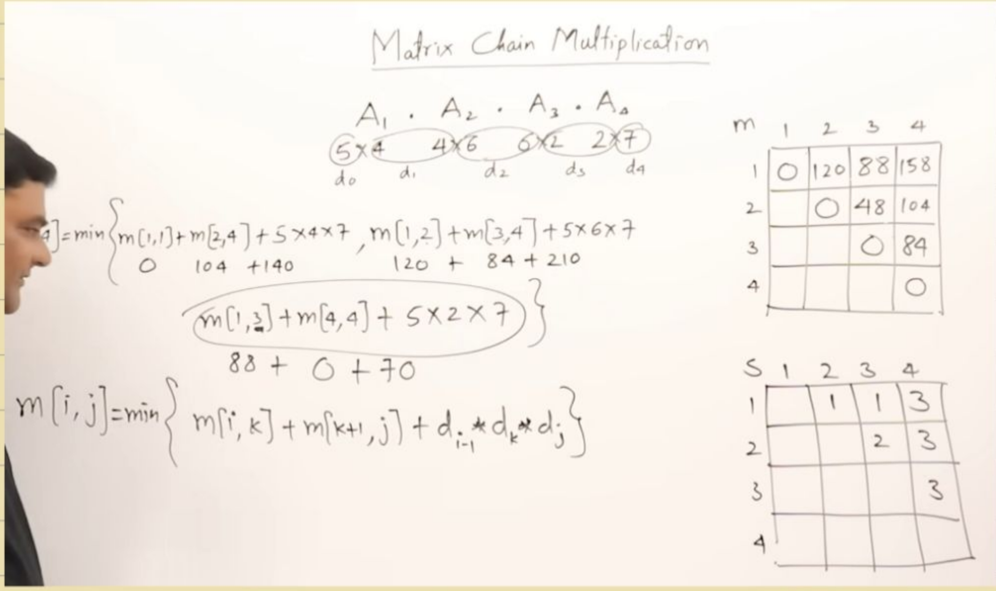


$$A^0 = \begin{array}{c|cccc} & 1 & 2 & 3 & 4 \\ \hline 1 & 0 & 3 & \infty & 7 \\ 2 & 8 & 0 & 2 & \infty \\ 3 & 5 & \infty & 0 & 1 \\ 4 & 2 & \infty & \infty & 0 \end{array}$$

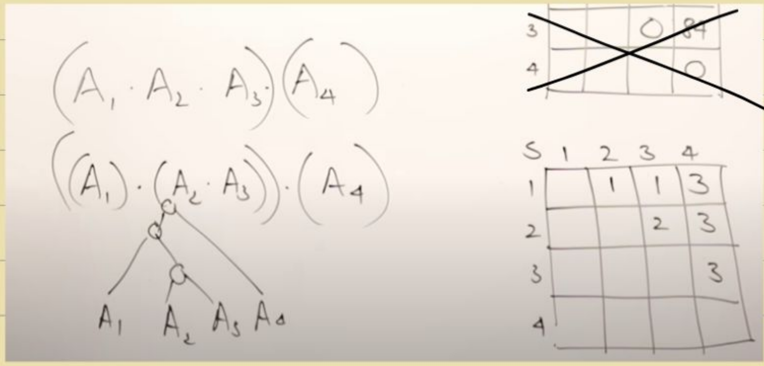
$$A^k[i,j] = \min\{ A^{k-1}[i,j],\ A^{k-1}[i,k] + A^{k-1}[k,j] \}$$

```
for(k=1; k<=n; k++)
{
  for(i=1; i<=n; i++)
  {
    for(j=1; j<=n; j++)
    {
      A[i,j]=min(A[i,j], A[i,k]+A[k,j]);
    }
  }
}
```

- Matrix chain multiplication :

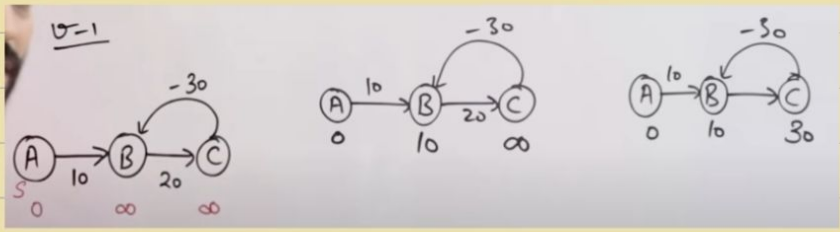


$O(n^3)$

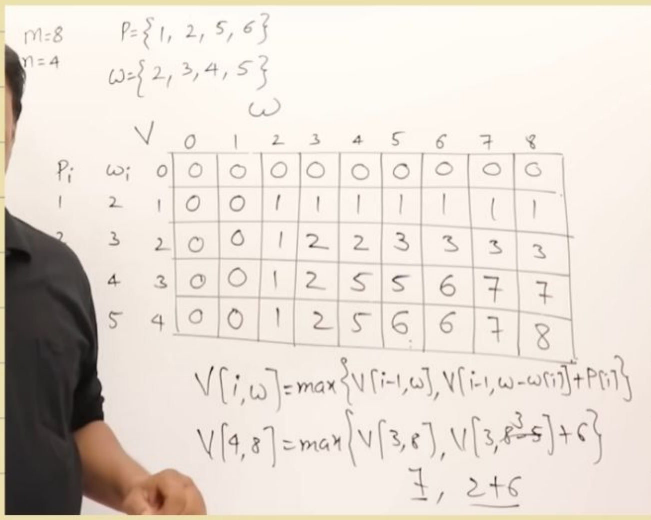


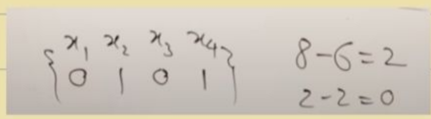
- Bellman Ford :

Relax '|V|-1' times

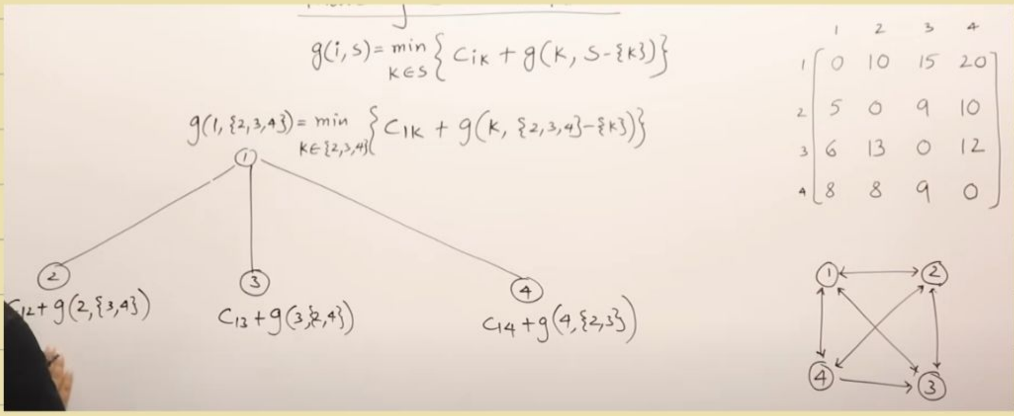


- 0/1 Knapsack :

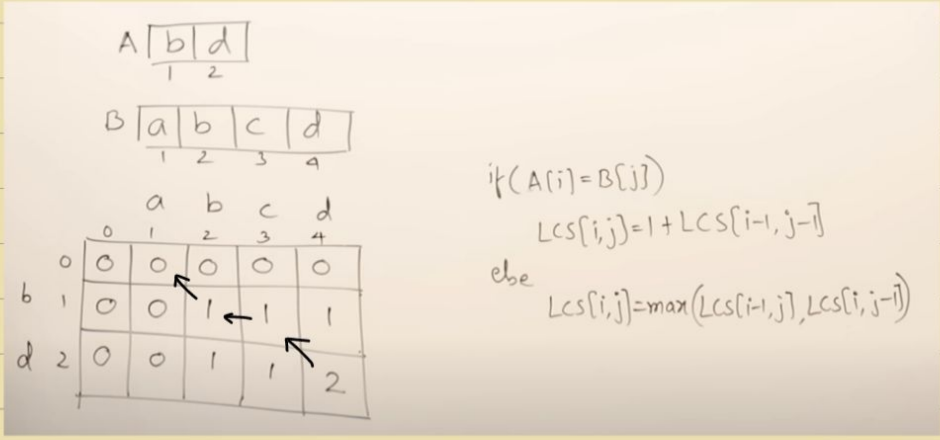




- Travelling Salesman Problem :

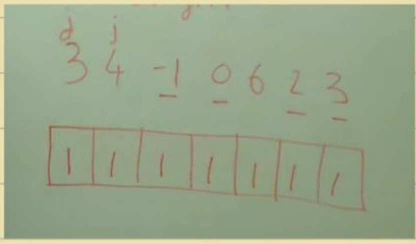


- Longest Common Subsequence :

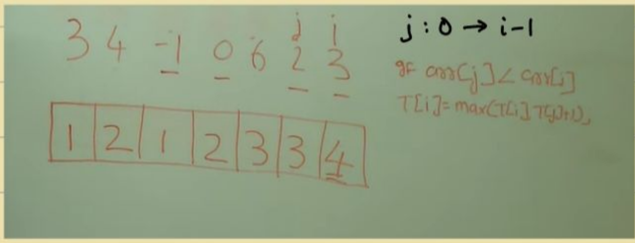


$O(m \times n)$

- Longest Increasing Subsequence :



$\Downarrow$



29/7

→ Algorithms and Complexity :

- A few wonders :
  - Follow your passion.

→ Graph Algorithms

→ Number Theoretic Algorithms

→ String Theoretic Algorithms

⋮

→ Computational Physics

→ Computational Chemistry

→ Computational Biology

→ Computational Mathematics

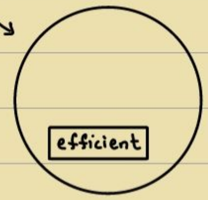
→ Computational Economics

→ Computational Music

→ Computational Finance

- "Interesting" Problems :
  - Tractability ?
    (feasible)
  - Efficiency ?
    (≡ Polynomial Time Algo)
    ↳ Convention

$O(2^n)$ ⟶ Restating the problem



Closed under Addition , Subtraction & Multiplication .

∵ Using an algorithm as a subroutine

Intuitively , Polynomial Time works.

Algo - runtime : No. of steps taken by the Turing machine

TM : Model of Computation

TM : Model of Complexity

T.M may not be able to simulate $O(2^n)$ efficiently.

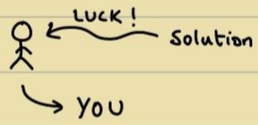
Worst case - Asymptotically - Polynomial : Efficient

- $\text{TIME}(n) = \{ L \mid L$ is decided by some DTM 'M' , in time $O(n) \}$

$$P = \bigcup_{k=0}^{\infty} \text{TIME}(n^k)$$

↳ Polynomial class ⟶ Interesting

i.e. Feasible and Efficient solution can exist.



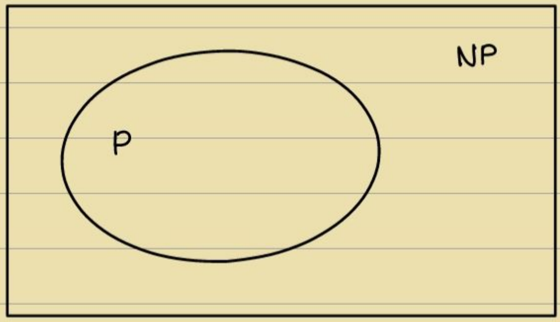
$x \in L$: Proof / Witness certificate

- CLIQUE = $\{ \langle G, k \rangle \mid G$ has a clique of size $k \}$
  - Trial and Error strategy (Keep trying)

    No. of trials ↑

    Luck can play a role, i.e. when solution is given – checking is in P

    ∴ Interesting → NP



Clique $\in$ NP

Given the solution, can you recognise / decide whether the solution is correct or not ? i.e. How difficult it is to check.

If this problem is in P ⇒ main problem in P

i.e. If problem → Unintersting

But if we find that verifying solution for that problem → Interesting

Then, Problem → Interesting

If problem is beyond NP ⇒ even less interesting but can be turned into even more interesting.

i.e. even if solution is given, difficult to verify solution.

e.g. Given $n \times n$ chessboard & Given winning strategy for winning every chess game in that board, as $n \to \infty$ ⇒ Outside NP

<u>Intuition</u> :

- Needle & Haystack

  If you're finding needle in the haystack :

  If you know what a needle is ⇒ lift it and verify : NP

  If you can't differentiate needle from haystack in Polynomial time ⇒ You don't know you picked needle or not : Beyond NP

  If Interaction with alien / Randomness (Alien has $\underbrace{\text{proof}}_{\text{can be verified}}$) : IP - Space

Complexity Theory → Hard (P, NP)

Beyond Complexity Theory,

Computability Theory → Really Hard (R, RE)

Open Problems :

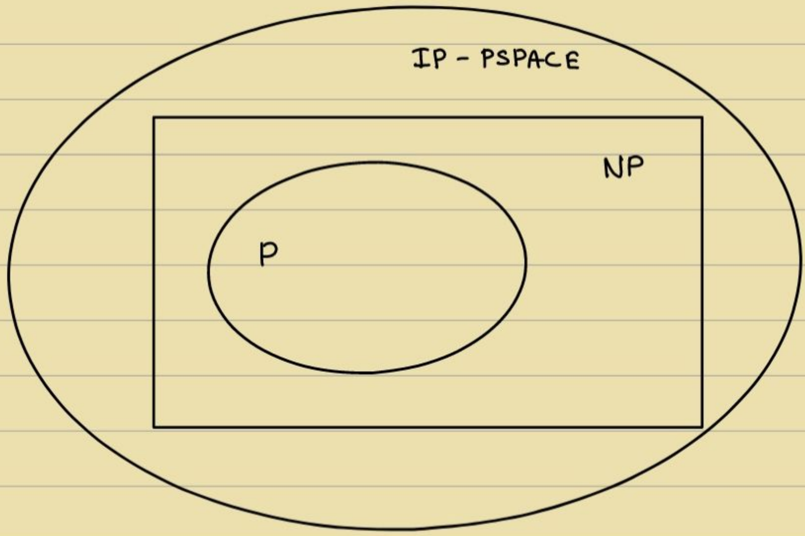
Is NP different from P ?

Is NP different from IP ?

- $\text{TIME}(n) = \{ L \mid L \text{ is decided by some NDTM 'M', in time } O(n) \}$

$$NP = \bigcup_{k=0}^{\infty} \text{NTIME}(n^k)$$

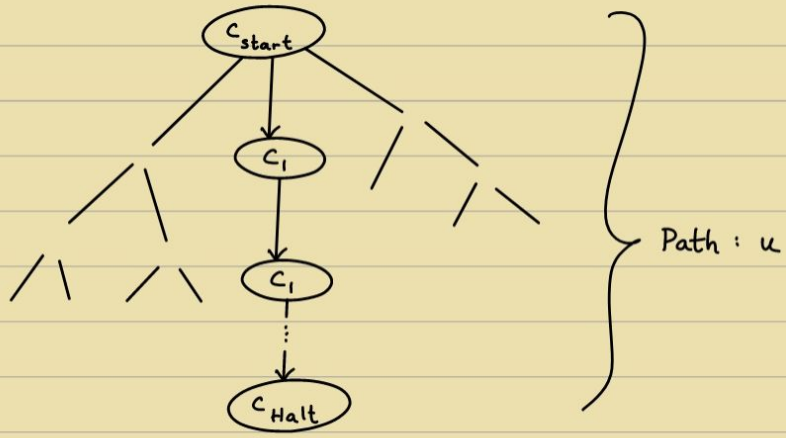
If searching for a needle in a haystack, if you're lucky you pick the needle, but the problem of finding whether you pick the needle or hay should be easy.



Verifier :

If ∃ path to solution : ACCEPT

If all computational paths are not accepting : REJECT



$L(M)$: M accepts x

M decides L if $L(M) = L$

Got the Needle, Then able to verify in P

$L(V)$: $\{x \mid \exists u \ V(x,u) \text{ accepts}\}$

V verifies L if $L(V) = L$

Efficient verifier : Given $x \Rightarrow$ Quickly check whether $x \in L$

A class of all polynomial time verifiers $\longrightarrow$ P (Intersting)

But P : class of all Polynomial time NDTMs (Accept path according to V)

If $\exists$ NDTM for a problem $\Rightarrow$ $\exists$ Polynomial time algo

NP = $\{L \mid L$ has an $\underbrace{\text{efficient}}_{\text{In Polynomial Time}}$ verifier$\}$

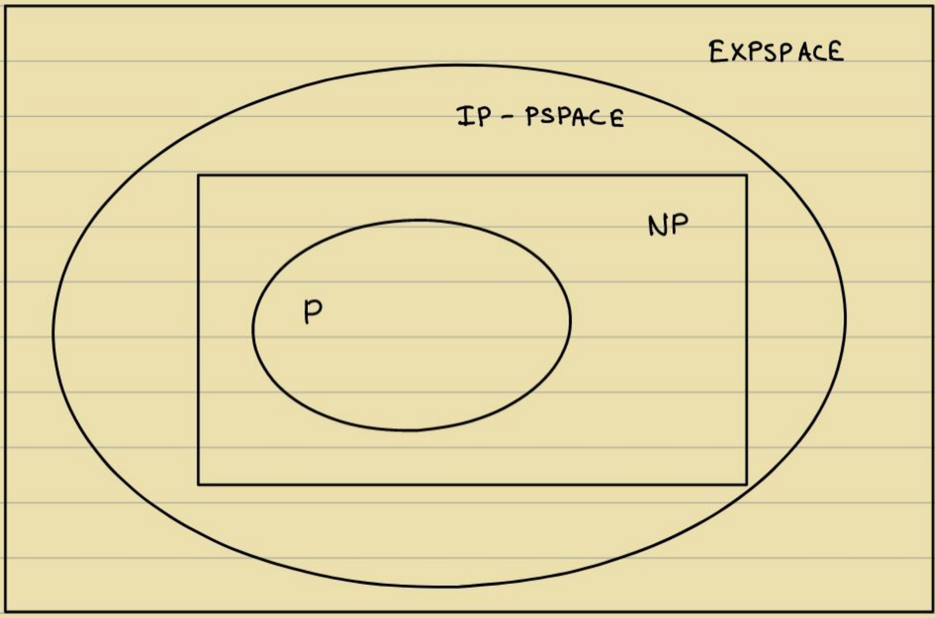
Interest based on luck

If $\exists$ Polynomial time NDTM $\Rightarrow$ $\exists$ Polynomial time verifier ($\exists$ a path u)

$\exists$ Polynomial time verifier $\Rightarrow$ $\exists$ Polynomial time NDTM ($\exists u$: $L(V)$ accepts)

Guess u & run

$\left.\begin{array}{l}\text{For Authorised People} \Rightarrow \text{Problem is easy} \\ \text{For non-Authorised People} \Rightarrow \text{Problem is Hard}\end{array}\right\}$ Security



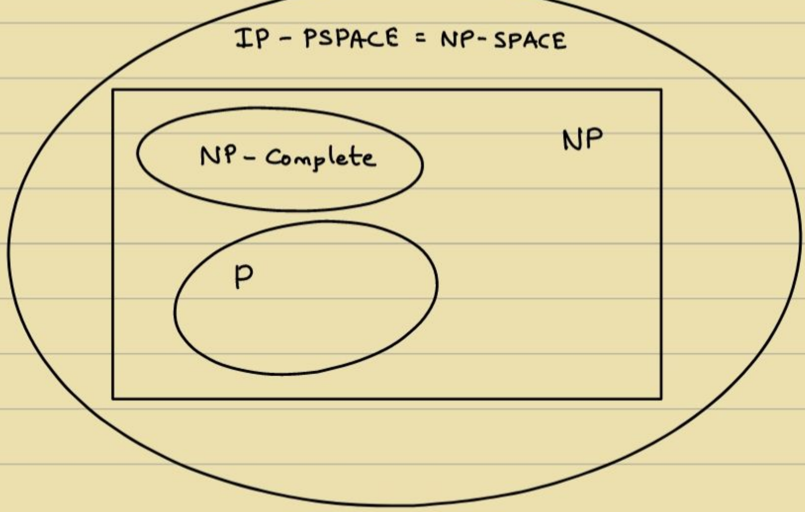
Complexity 'zoo'

- Efficiently & easily pose hard question $\leftarrow$ Security Tenet

only when $P \neq NP$

$\because$ Easily Verify $\Rightarrow$ Easily decide

$\because$ If $\exists u \longrightarrow$ Problem becomes easy (u : known)

- 

- <u>Reducability</u> :

  If $\exists$ Mapping Reduction
  $L_1 \leq_p L_2$
  if $\exists f$,
  $x \in L_1 \Leftrightarrow f(x) \in L_2$

  $\Rightarrow$

  If $\exists$ Polynomial (time) reduction
  $L_1 \leq_p L_2$
  if $\exists f$ (Computable in Poly-time),
  $x \in L_1 \Leftrightarrow f(x) \in L_2$

- <u>NP - Complete</u> :

  If $\exists$ efficient solution for any NP problem, then NP = P

  $NPC = \{ L \in NP \mid \forall L' \in NP, L' \leq_p L \}$

  i.e. using L, you can solve every other L'.

  - <u>Theorem</u> : Boolean satisfiability is NP-Complete.

    SAT : $\{ \phi \mid \phi$ is a CNF boolean formula that is not a fallacy $\}$

    (<u>H.W</u> : Prove CLIQUE $\in$ NP - Complete)

  <u>Note</u> : If $\exists$ an efficient solution to solve CLIQUE, Then every problem in NP can be efficiently solved $\Rightarrow$ P = NP , i.e. all-in-one problem

  <u>Recall</u> : General Purpose computing is possible because of existance of Universal Turing machine, which can take a TM itself as input, i.e. it's possible to build one hardware which can simulate every other hardware using the software.

NPC is used to show that $\exists$ problems that are hard, i.e. unsolvable.

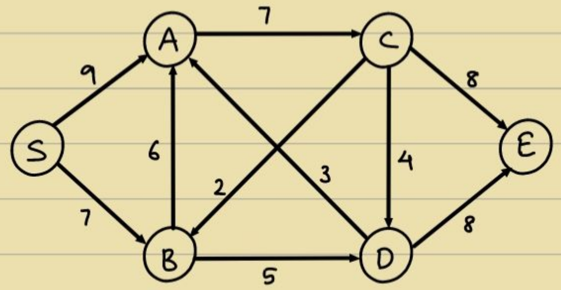
$\because$ NPC is used to show CLIQUE is hard. If not, every problem is solvable.

$\therefore$ Approximation Algorithms, Randomisation Algorithms and Quantum Algorithms are used instead to deal with hard Problems.

6/10

→ Network Flows

$G = (V, S)$



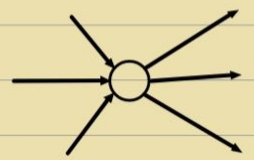
e.g. Water pipeline with flow capacities

Q) What's the maximum flow from S to E ?

i.e. Objective :

$$\text{maximise} \sum_{\substack{e=(s,u) \\ \in E}} f(e)$$

- Conserved :



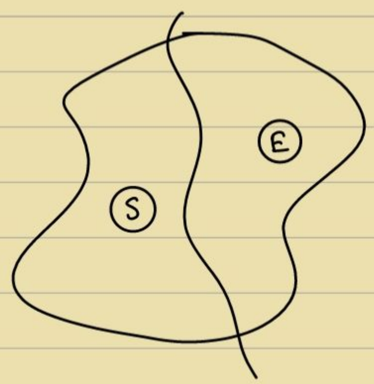
flow , $f : E \longrightarrow \mathbb{R}$

$\forall e \in E \, , \; f(e) \le c(e)$

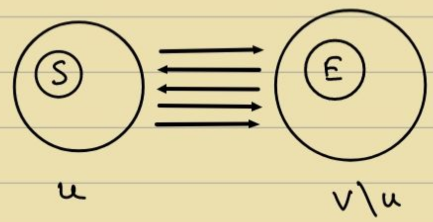
Total - in = Total - out

Theorem : Maximum Flow = Minimum Cut

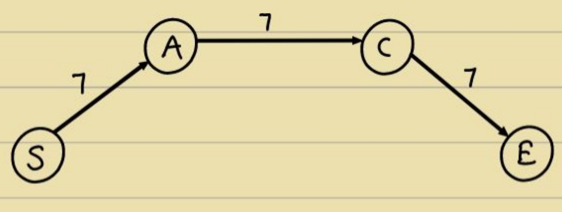
Max. flow ≤ Min. cut

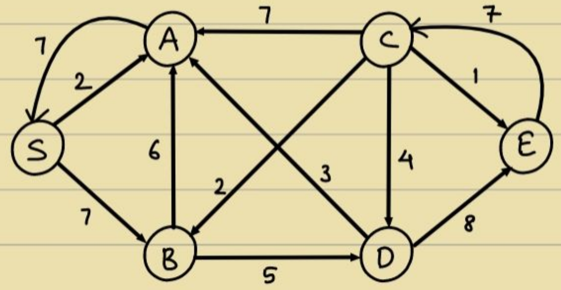


≡



- FF Algo :
  - Current Flow
  - Augment the Flow (find an augmenting path in Residual Graph)



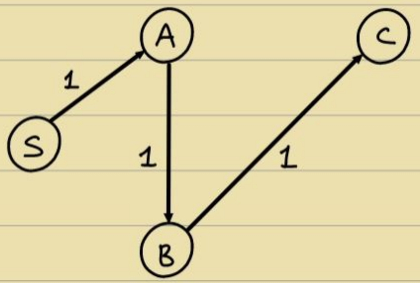


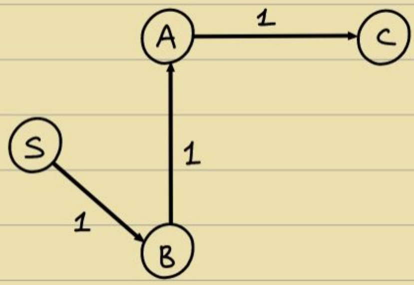
Given current flow f

Residual Graph capacities : $c(e) - f(e)$ (Original direction) S → E

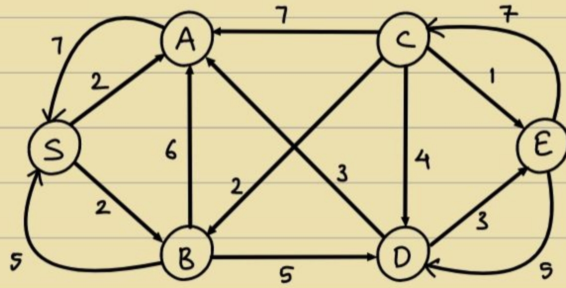
(u, v)

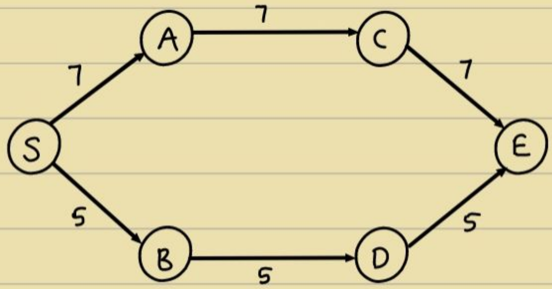
$f(e)$ (Reverse direction)



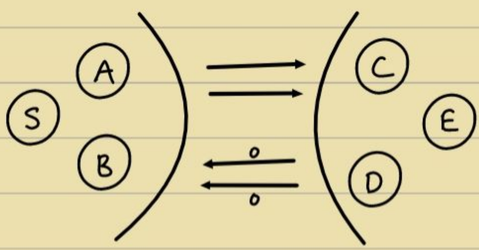


Now,

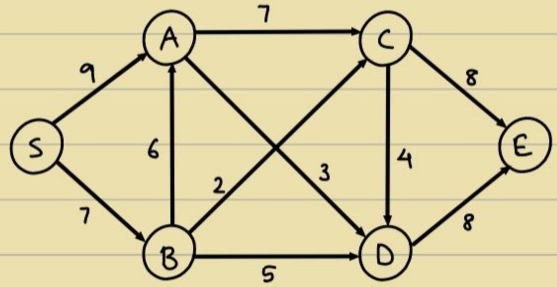




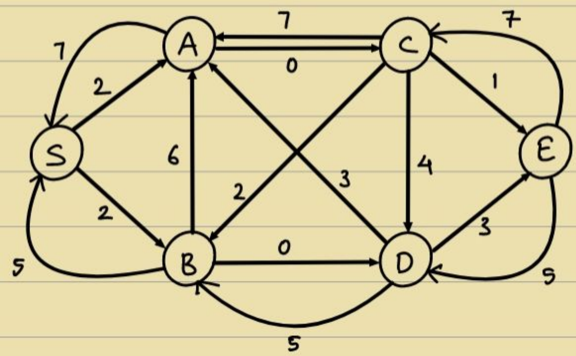
Max = 12

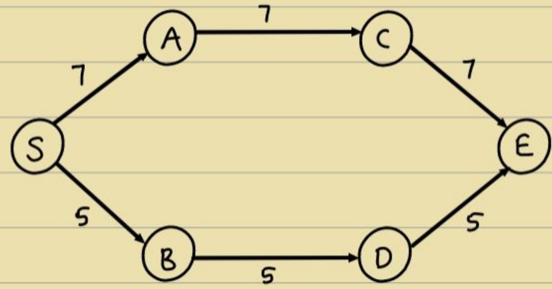


Ex.

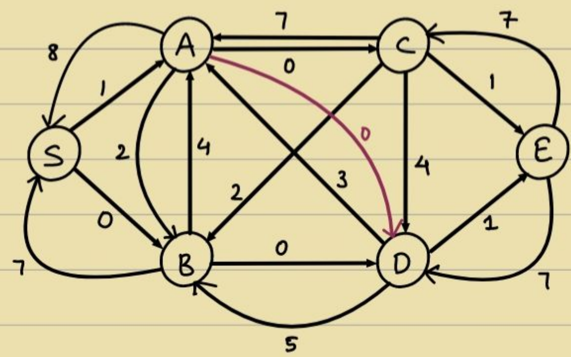


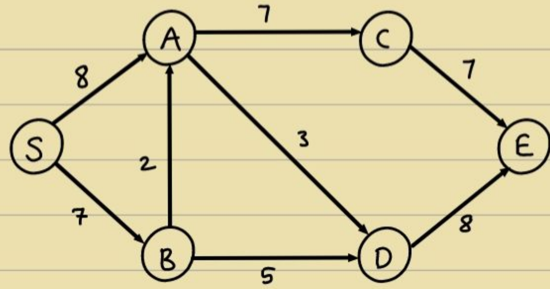
Residual graph:



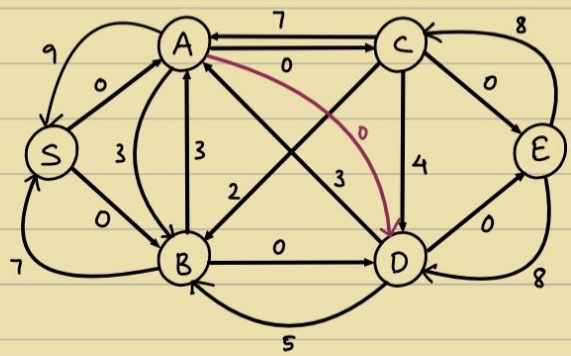


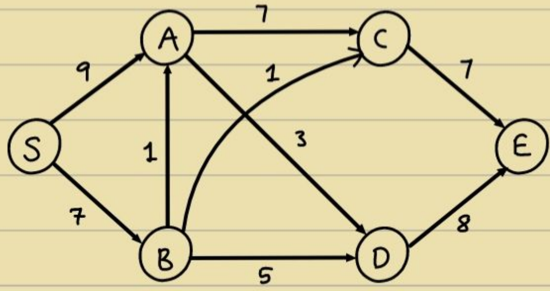
3rd Augmentation:





Final Augmentation:





Max : 16

- **Review** :

  Input : $G \equiv (V, E)$

  Edge weighted

  Output : $f: E \to \mathbb{R}$ [Max flow]

  Conserved & Valid / feasible

  Algo : Augment current flow by adding some augmenting path in / from the residual Graph

$$u \xrightarrow{c(e) - f(e)} v$$
$$v \xleftarrow{f(e)} u$$

- Efficient Algorithm
  - But not efficient if capacity of path is extremely large.
  - Simplex algorithm for linear programming ← Adding 1s ⇒ Exponential (Maybe)

  But, Ford - Fulkersion algorithm ← Jumps ⇒ Polynomial

13/10

→ NP - Completeness and Approximation Algorithms :

- Minimal Set Cover :

  I/P : A set S & family of subsets of S, $F \subseteq 2^S$ (Powerset of S)

  O/P : The least no. of elements that cover S (i.e. Union of s)

e.g. $S = \{1, 2, 3, 4\}$

$F = \{\{1,2\}, \{1,2,3\}, \{1,4\}, \{2,3\}, \{2,4\}, \{4\}\}$

Minimum set cover : 2 bcs $(\{1,2,3\}\{4\}, \{1,4\}\{2,3\})$

Facts :

① Minimum Set Cover is NP - Hard.

(How to show this ?)

② There is a greedy strategy that approximates the answer within a factor of $\log|S|$. ← Approximation Ratio

i.e. Computed answer $\leq \log|S|$ [OPTIMUM]

Greedy Algorithm of min-set cover :

Choose maximum |element| set and keep choosing max-sets till all elements covered.

for above e.g. ① $\{1, 2, 3\}$

② $\{1, 2, 3\}\{4\} \longrightarrow 2$, Here it's optimal

But it need not be like this always

Greedy Algo :

Always choose the next element of IF that covers the maximum number of not-yet-covered elements of S.

Theorem :

Approximation ratio is $\log|S|$

(of greedy not-cover)

i.e. optimal answer (# of sets that cover S) $\leq \log|S|$

To solve : $t \leq \log|S| \cdot k$ [OPTIMUM]

Proof :

Let n_i be the no. of uncovered elements in S after i iterations of the greedy algo

$n_0 = S$	After i iterations,
$n_t < 1$	n_i : remain uncovered
	: $\exists k$ sets that cover it

Using PHP :

$\exists$ a set that covers $\geq \frac{n_i}{k}$

$$\Rightarrow n_{i+1} \leq n_i - \frac{n_i}{k}$$

$$\Rightarrow n_{i+1} \leq n_i\left(1 - \frac{1}{k}\right)$$

$$n_i \leq n_0\left(1 - \frac{1}{k}\right)$$

$$\Rightarrow n_i \leq |S|\left(1 - 1/k\right)$$

$$\Rightarrow 1 - x \le e^{-x} \qquad [0 < x < 1]$$

$$\Rightarrow \left(1 - \frac{1}{k}\right) \le e^{-1/k}$$

$$\Rightarrow n_i \le |S| \cdot e^{-1/k}$$

$$@ \; i = k \cdot \ln|S| \Rightarrow n_{k \cdot \ln|S|} \le e^{\frac{-k\ln|S|}{k}} = 1$$

$\therefore \; t \le \ln|S| \cdot k$

after $t = \ln|S| \cdot k$ iterations, greedy algo terminates.

→ **SAT** :

SAT = $\{ \phi \mid \phi$ is a satisfiable CNF Boolean formula $\}$

$\phi = (r \vee r \vee \dots r) \wedge (\dots) \wedge (\dots)$

Cook – Levis Theorem :

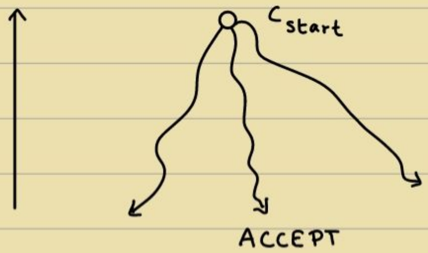
SAT is NP - Complete.

i.e. (a) SAT $\in$ NP

(b) $\forall L \in NP$ S.T. $L \le_p$ SAT

{ Karp Reduction :
→ Polytime mapping reduction }

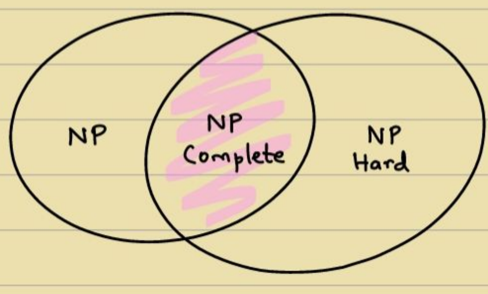
Note : $L \in NP$ means that $\exists$ NDTM 'N' that decides L in Poly time.



CNF ϕ_N

ϕ_N is satisfiable iff N has an accepting path.
{ NDTM form ⟶ Boolean form : Reduction }

$A \le_p B$ if $\exists$ poly-time computable f S.T. $\forall x \in \{0,1\}^*$,
$x \in A \Rightarrow f(x) \in B$



Th: CLIQUE is NP-Complete

Proof: Recall CLIQUE $\in$ NP

CLIQUE $= \{ \langle G, K \rangle \mid G$ has a clique of size $k \}$

To show: $\forall L \in NP$, $L \leq_p$ CLIQUE (CLIQUE is NP-Hard)

SAT $\leq_p$ CLIQUE

SAT $\leq_p$ 3-SAT [2-SAT is in P]

3-SAT $= \{ \phi \mid \phi$ is a satisfiable 3-CNF $\}$

3 CNF $\phi = (\cdot \vee \cdot \vee \cdot) \wedge (\cdot \vee \cdot \vee \cdot) \wedge \ldots$

i.e. exactly 3 literals per clause

e.g. $(x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee x_2 \vee \bar{x}_3) \wedge \ldots$

if there are more than 3, then reduce to 3 CNF:

i.e. $(x_1 \vee x_2 \vee x_3 \vee x_4)$

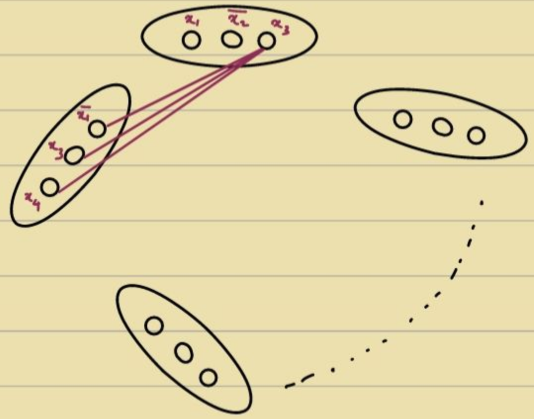
$= (x_1 \vee x_2 \vee x_5) \wedge (x_3 \vee x_4 \vee \bar{x}_5)$

Given 3 CNF: $\phi = (\cdot \vee \cdot \vee \cdot) \wedge (\quad) \wedge (\quad) \ldots \wedge (\quad)$

n variables

l clauses

Construct a graph G_ϕ:



$3l$ vertices

Add all non-contradictory edges across clauses.

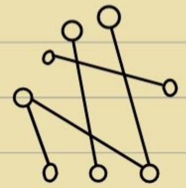
i.e. (a) Don't add edge in the same clause / node

(b) $\bar{x}_i$ and x_i should not be added

Does G_ϕ have a clique of size l ?

Satisfying assignment

$x_1, \ldots\ldots \overline{x}_\ell \longrightarrow \ell$ literals

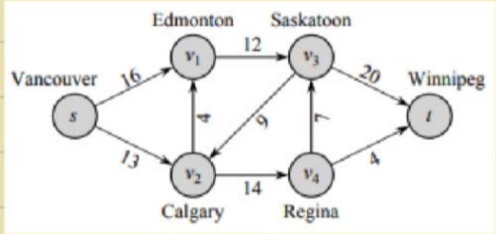


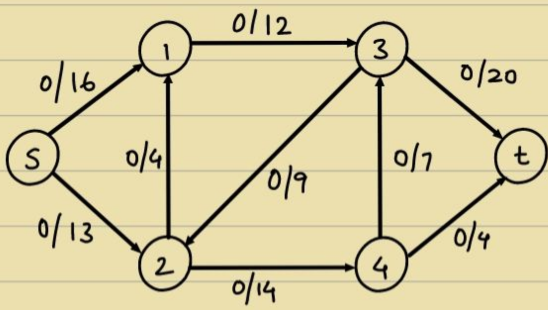
ℓ vertices

$\therefore$ CLIQUE is NP Hard.

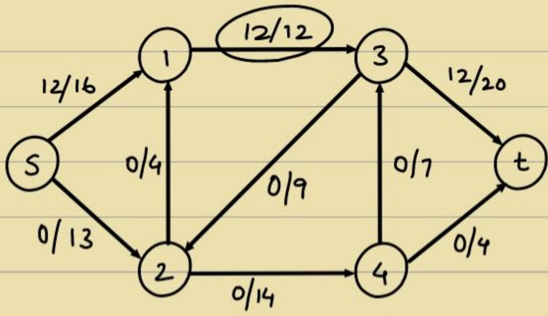
$\therefore$ CLIQUE is NP Complete. $[\because$ CLIQUE is also in NP$]$

(4)

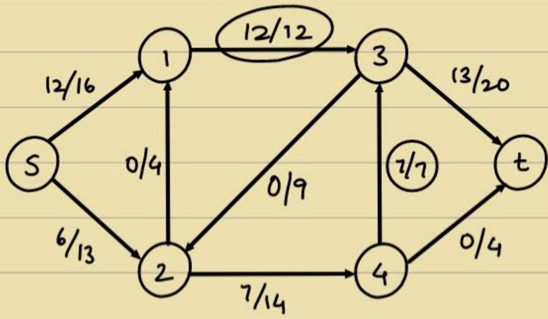




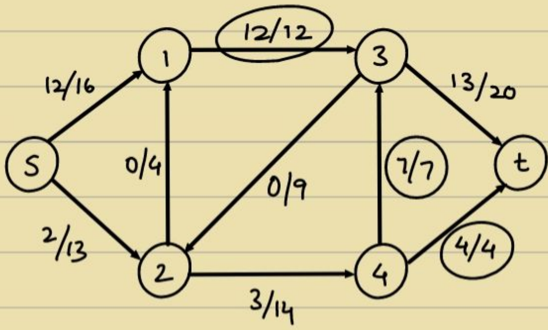
$s \to 1 \to 3 \to t$: 12



$s \to 2 \to 4 \to 3 \to t$: 7

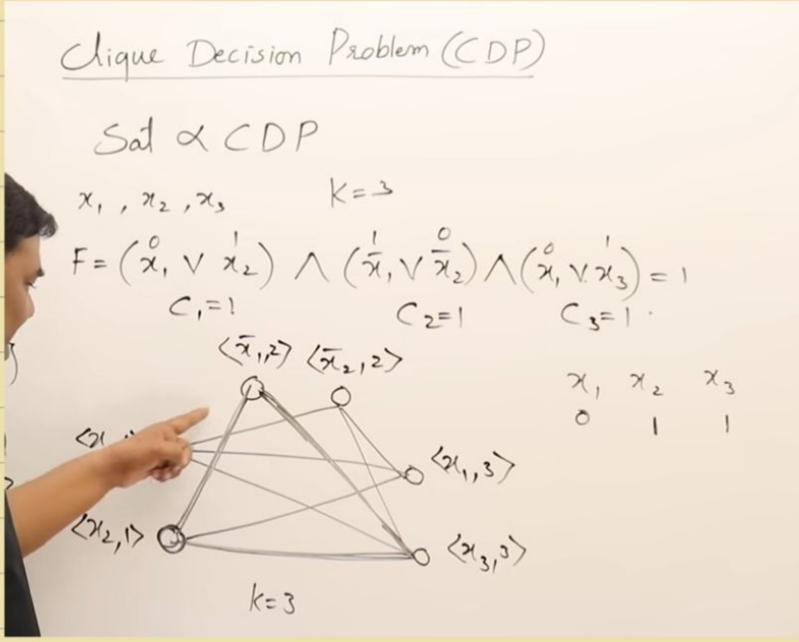


$s \to 2 \to 4 \to t$: 4



Maximum flow : $\underline{12 + 7 + 4 = 23}$

19)



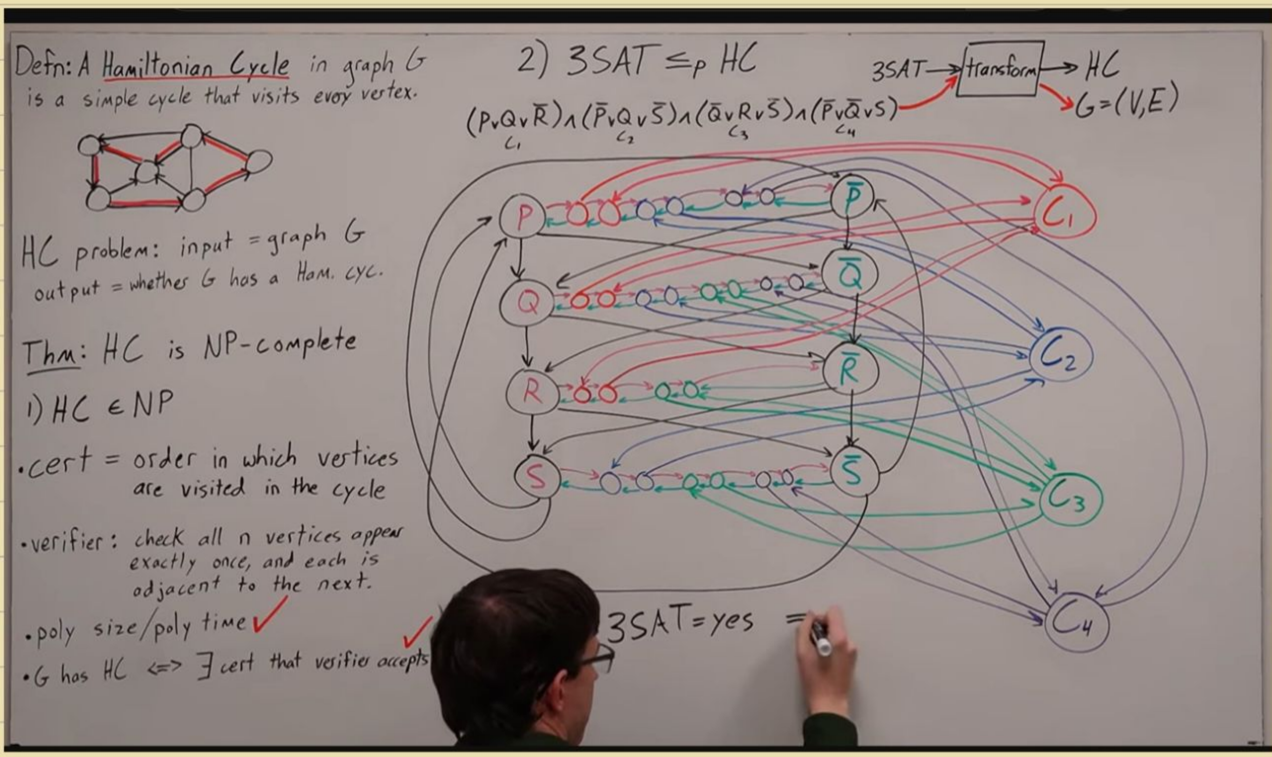
20)

Independent Set Problem Given a graph G and an integer K, does G contain an independent set of size K?

Proposition 8. The Independent Set Problem can be reduced to the Vertex Cover Problem in polynomial time.

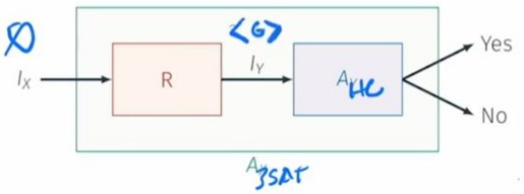
Proof. Let G be a graph and integer K be an instance of the independent set problem. Let n be the number of vertices of G and m be the number of edges.
Graph G has an independent set of size K if and only if G has a vertex cover of size $n - K$.
The mapping (G, K) to $(G, n - K)$ is a reduction of the independent set Problem to the vertex cover Problem and copying G takes $\Theta(n + m)$ time which is a polynomial in n and m. □

21)



(OR)

- Directed Hamiltonian Cycle is in *NP*: exercise
- Hardness: We will show 3-SAT $\leq_P$ Directed Hamiltonian Cycle



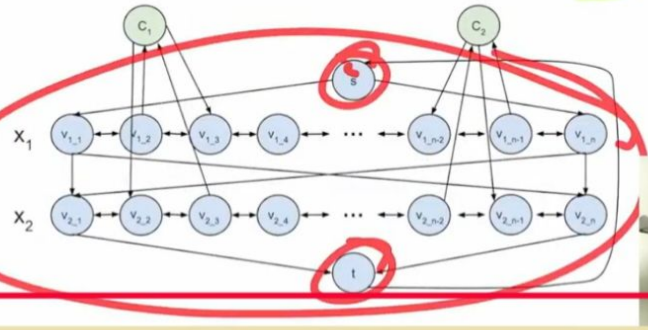
How do we handle clauses?

$$f(x_1, x_2) = (x_1 \vee \overline{x_2}) \wedge (\overline{x_1} \vee x_2)$$

0 0
0 1 c_1
1 0 c_2
1 1

What if the expression has multiple clauses:

1 →
0 ←



23) **Vertex Cover Problem** Given a graph G and an integer M, does G have a vertex cover of size M?

Proposition 10. The Vertex Cover Problem can be reduced to the Set Cover Problem in polynomial time.

Proof. Let graph $G = (V, E)$ and integer M be an instance of the vertex cover problem. Let $U = E$ and let $S_i = \{e_j : v_i \text{ is incident on } e_j\}$ and C be the collection $S_1, S_2, \ldots, S_n$. Graph G has a vertex of size M if and only if U has a set cover of size M.
Mapping (G, M) to (U, C, M) is a reduction of the vertex cover problem to the set cover problem and constructing U and C takes $O(mn)$ time which is a polynomial in m and n. □

22)

[redacted]

① LP is NP (easy)

check such path exists if some one gave you a path

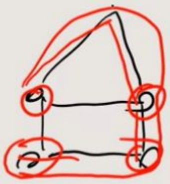
is in polynomial time.

② LP is NP-hard   Hamiltonian path $\leq$ LP

[redacted]

Hamiltonian path: $G=(V,E)$

there is a path connecting all vertex without repeating ~~vertex~~ vertices



pf:

Given $G=(V,E)$

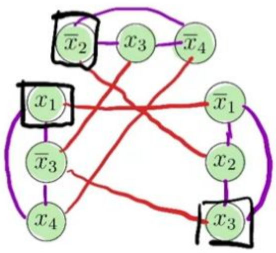
$\longrightarrow G=(V,E),\ K \overset{?}{=} |V|-1$

IS:

graph has an independent set, then φ is satisfiable,

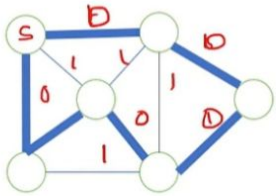
$$\varphi = (x_1 \vee \overline{x}_3 \vee x_4) \wedge (\overline{x}_2 \vee x_3 \vee \overline{x}_4) \wedge (\overline{x}_1 \vee x_2 \vee x_3)$$

1. Create a vertex for each literal.
2. Connect each literal to the other two literals in the same clause.
3. Connect each literal x_i to $\overline{x}_i$.



TSP :

Travelling Salesman ∈ NP
Given a sequence of vertices. The verification algorithm checks that this sequence contains each vertex exactly once, sums up the edge costs, and checks whether the sum is at most k.



Hamiltonian Cycle → Travelling Salesman

Hamiltonian Cycle ≤p Travelling Salesman
Let G=(V, E) is an instance of Hamiltonian Cycle. An instance of TSP is constructed as,
Form a complete graph G'=(V, E'). Where E'={(i, j): i, j ∈ V and i ≠ j}
Cost is defined as

$$c(i,j) = \begin{cases} 0 \; if \; (i,j) \in E, \\ 1 \; if \; (i,j) \notin E \end{cases}$$

The graph G has a Hamiltonian cycle if and only if graph G' has a tour of cost at most 0.

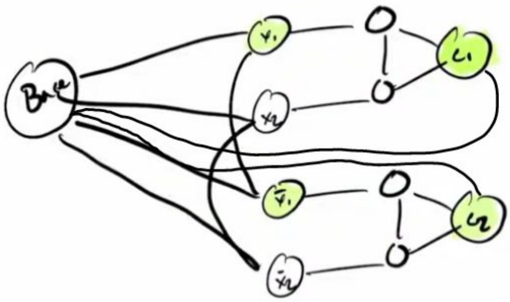
TSP $\leq_p$ Hamiltonian

3 Colouring

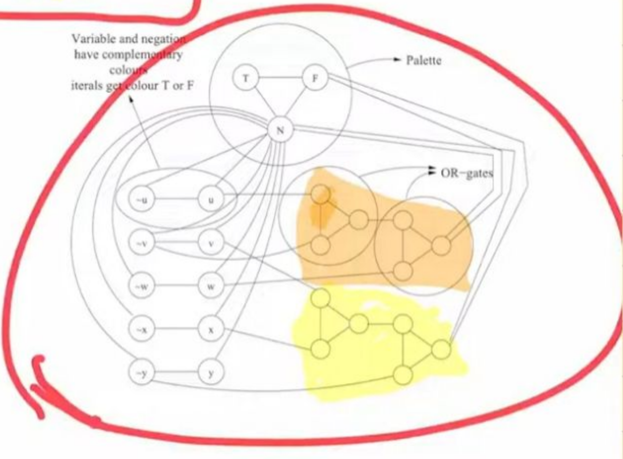
Next we need a gadget that assigns literals. Our previously constructed gadget assumes:

- All literals are either red or green.
- Need to limit graph so only x_1 or $\overline{x_1}$ is green. Other must be red

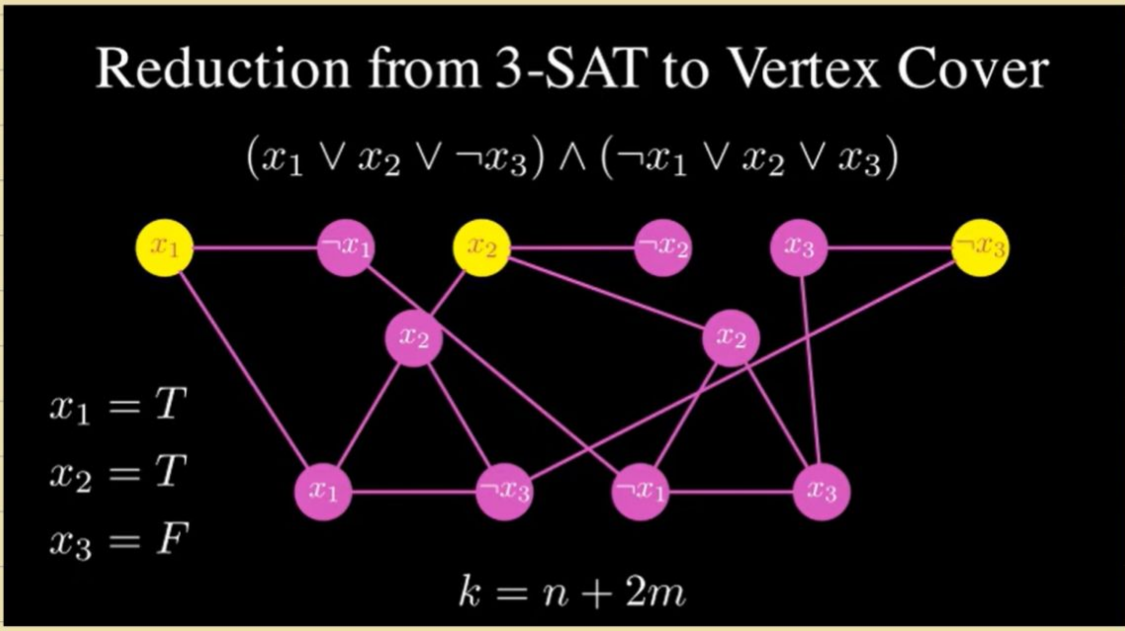
$\varphi = (x_1 \vee x_2) \wedge (\bar{x}_1 \vee \bar{x}_2)$



Example
$\varphi = (u \vee \neg v \vee w) \wedge (v \vee x \vee \neg y)$



Vertex Cover:



Approximation Algo's :

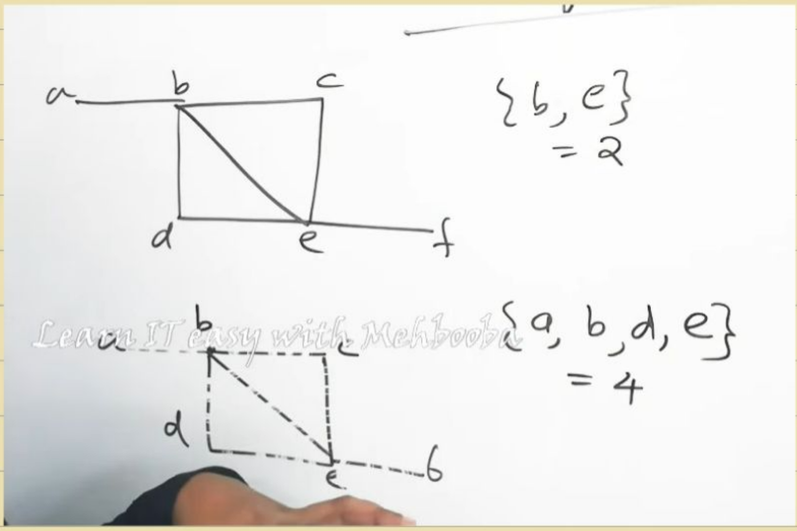
Knapsack :

Approximation Schemes -

- This algorithm generates all subsets of k items or less & for each one that fits into knapsack it adds the remaining items as greedy alg would do (ie, in non increasing order of v/w ratio)
- The subset of highest value obtained in this-fashion is returned as alg op.

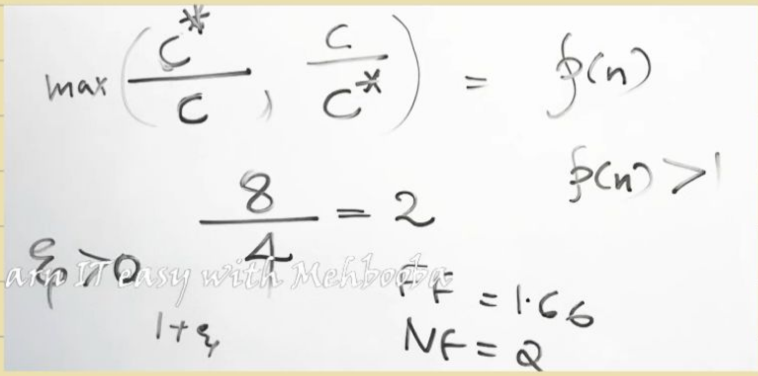
$$\frac{f(s^*)}{f(s_a^k)} \leq 1 + \frac{1}{k} \rightarrow \text{Accuracy Ratio.}$$

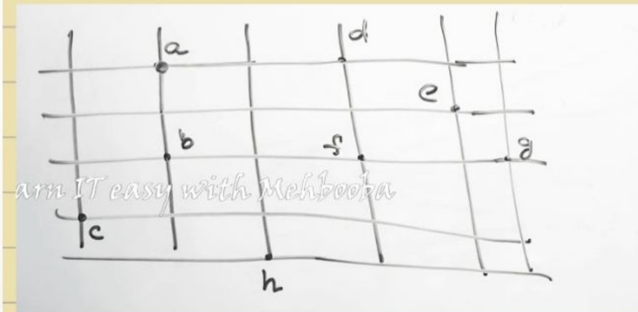
Vertex Cover :



← 2- approximation Algo

TSP :



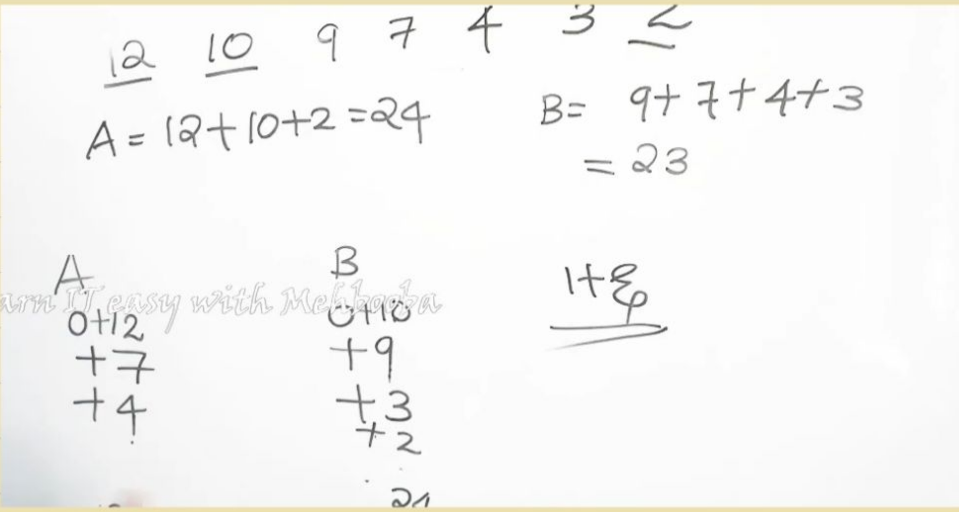


Prims Algo - MST

Preorder Traversal

Hamiltonion Cycle (Preorder)

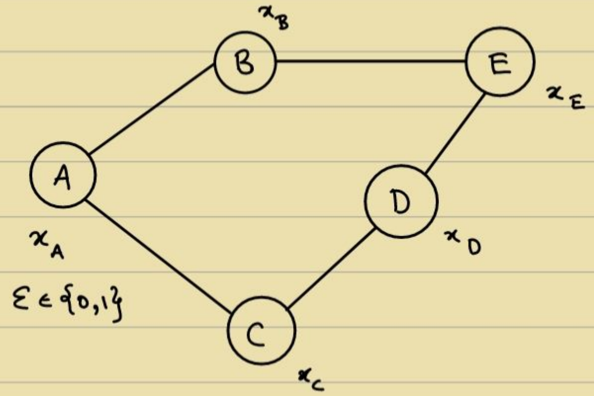
Minimum Partitioning:



30/10

→ **Byzantine Agreement**

Flow of distributed Algorithms



A protocol S.T.

→ All non-faulty outputs are the same [Agreement]

→ The Agreed output must be the same party's input. (non-faulty)

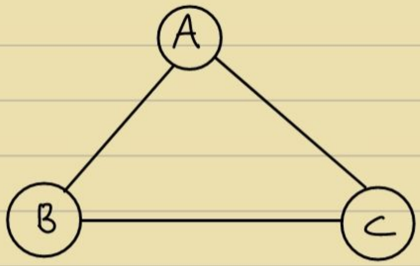
Faulty party : Modifies the code (deligate to it)

Theorem :

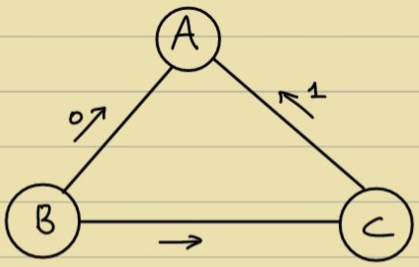
1 out of 3

Synchronous network

Byzantine Algorithm is impossible



Proof :

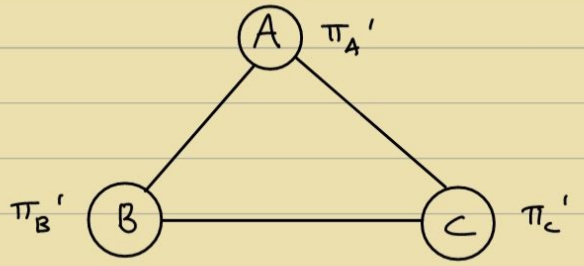


Solution ∃ for signed messages only

Claims Sent 0

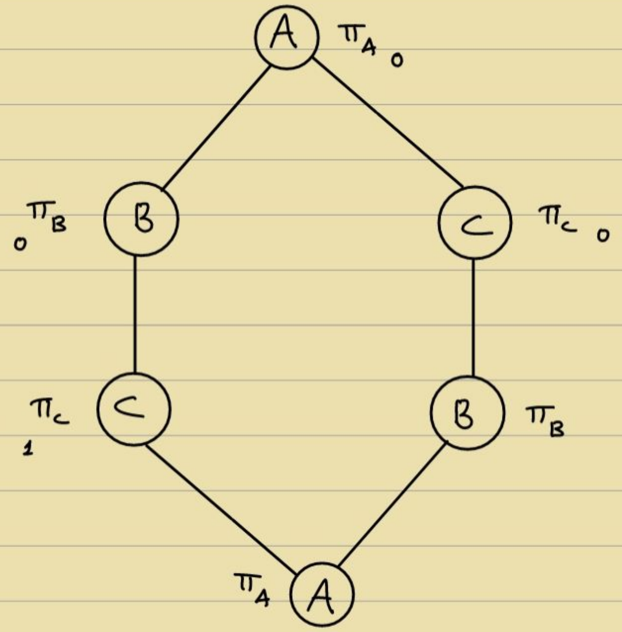
Claims Recieved 1

Intuition : If truth is known to only 2 people in the world and they decide to contradict each other $\Rightarrow$ The truth is burried.



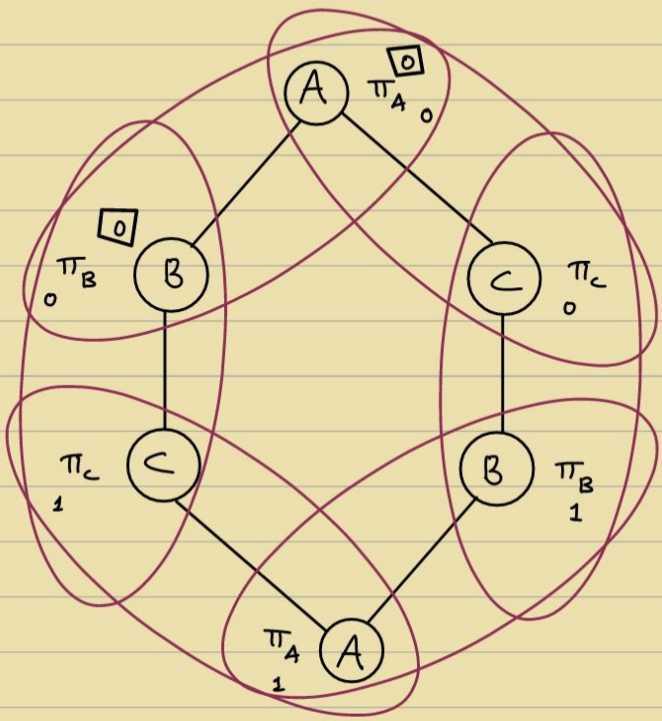
<u>Proof</u> :

Suppose a Protocol for BA exists



Proof by Contradiction
Adversial strategy

We don't know who is faulty.



B is split into 2 nodes.
AC doesn't know which network is it part of.

Never the same output

$\Rightarrow \Leftarrow$

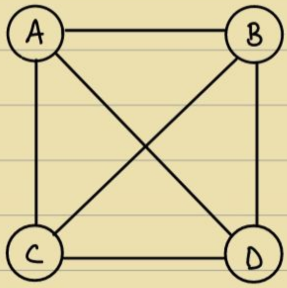
Theorem :

N/ω of n nodes

↑

Synchronous completely connected

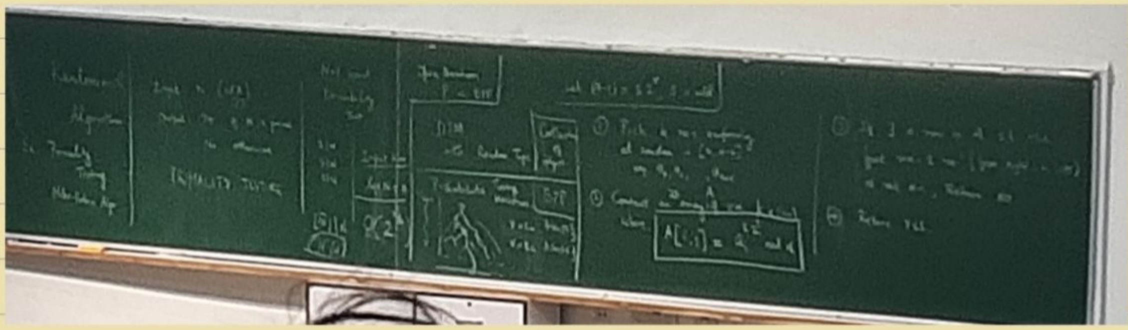
Up to t nodes are Byzantine faulty,
BA is possible if and only if $n > 3t$
& Graph is $(2t-1)$ connected



$n = 3t + 1$

- **Open problem :**
  Given a Graph $G(V, E)$ s.t. G is $(2t+1)$ connected and $|V| \geq 3t$, whats the minimum no. of rounds required for BA

3/11



Let $n = 15$, $k = 3$

$14 = 7 \times 2^1$

$r = 1$
$s = 7$

$a_0 = 3$

$a_1 = 7$

$a_2 = 11$

12	9

NO

Let $n = 7$, $k = 3$

$6 = 2^3 \times 1$, $r = 1$, $S = 3$

$a_0 = 3$

$a_1 = 4$ $\quad 4^6 \bmod 1$ $\quad\quad 4^6 \bmod 7$

$a_2 = 5$

6	1
1	1
6	1

Yes!

If $n \rightarrow$ Prime $\Rightarrow$ Always output YES

$n \rightarrow$ Composite $\Rightarrow$ Probability of output NO is atleast $1 - \frac{1}{2^k}$

<u>Theorem</u>: If n is prime $\Rightarrow$ Always output YES

<u>Proof</u>:

From Fermat's Theorem,

$\forall a \in (0, n-1)$, $a^{N-1} \bmod N = 1$

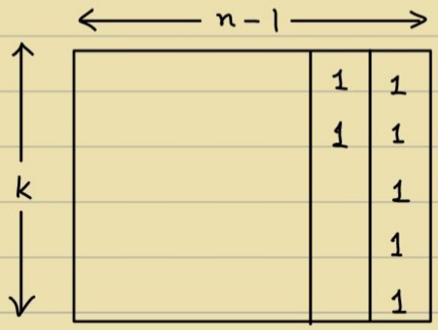
$(\because N \rightarrow \text{Prime})$

Intuition: $(a \quad 2a \quad 3a \quad \ldots \quad (n-1)a)$

$(1 \quad 2 \quad 3 \quad \ldots\ldots\ldots \quad N-1$

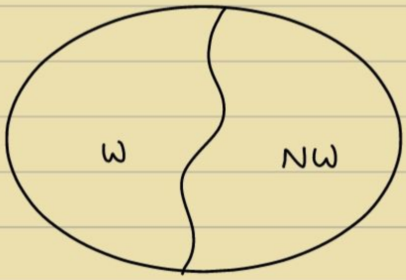
$$\prod_{i=1}^{n-1} a^i = \prod_{i=1}^{n-1} i \bmod n$$

$$a^{n-1} = 1$$



<u>Theorem</u>: If n is composite $\Rightarrow$ Always output NO with probability $\geq 1 - \frac{1}{2^k}$

Proof:



W : Witnesses

NW : Non - Witnesses

If $|W| \geq |NW|$

$f : NW \longrightarrow W$

one - one

Chinese Remainder Theorem :

$Z_{12} \cong Z_1 \times Z_2$

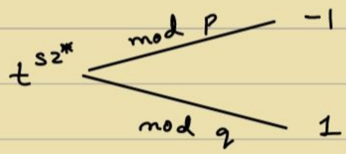
$a \longrightarrow a$

$N = pq$ $\qquad \gcd(p, q) = 1$

let $t \in [0, n-1]$ s.t

$t \equiv t \pmod 1$

$t \equiv 1 \pmod N$



Let $a \in NW$

$at \in W$

$f : NW \to W$

$f(x) = tx \bmod N$

$\Rightarrow |W| \geq |NW|$

6/11

Introduction to Cryptography

RSA Algorithms

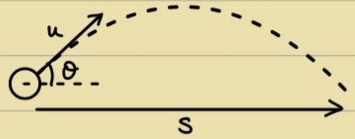
10/11

Distributed Algorithms Textbook

- Projectile Algorithms :

$$s = ut + \frac{1}{2}at^2$$

→ Quadratic in time



- Three polarizers — Science Experiment
  (Coolphysicsvideos Physics)

[Superposition principle]

→ Quantum Algorithms :

Postulate #1 : (Superposition)

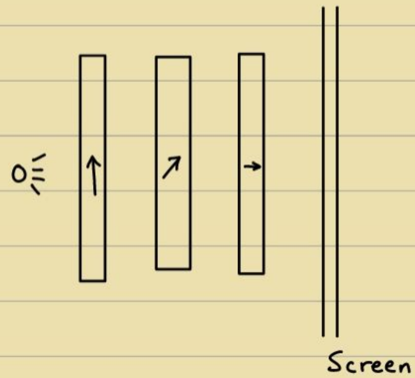
Associated with every quantum is a state vector.

$$|\psi\rangle = \alpha\,|\uparrow\rangle + \beta\,|\rightarrow\rangle$$

β → Not passing through

α → Passing through

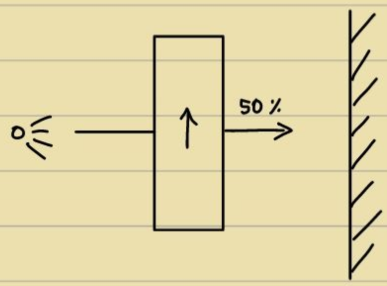
$$\therefore \begin{pmatrix} \alpha \\ \beta \end{pmatrix}, \quad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1$$



Postulate #2 : (Measurement and Collapse)

Probability of an outcome is $|\text{amp}|^2$

$|\alpha|^2$: Probability of outcome $|\uparrow\rangle$



$$|\psi\rangle = \alpha\,|\uparrow\rangle + \beta\,|\rightarrow\rangle$$

$$|\nearrow\rangle = \frac{1}{\sqrt{2}}\,|\uparrow\rangle + \frac{1}{\sqrt{2}}\,|\rightarrow\rangle \qquad \left[\because \left(\frac{1}{\sqrt{2}}\right)^2 = \frac{1}{2}\right]$$

$$|\uparrow\rangle = \frac{1}{\sqrt{2}}\,|\nearrow\rangle + \frac{1}{\sqrt{2}}\,|\nwarrow\rangle$$

Postulate #3 : (Evolution)

Unitary transformations on the state vector

$$U[\ \ ] = [\ \ ]$$

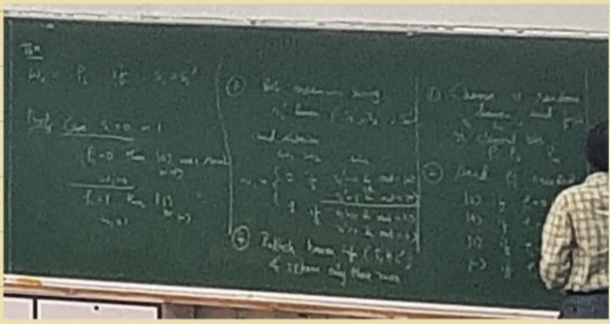
Unitary Matrix

$$[\,|\uparrow\rangle = \alpha\,|\nearrow\rangle\,]$$

$$U^{-1} = U^{T}$$

Conjugate Transform

17/11



→ **Popular Quantum Gates :**

I - gate : $\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$

X - gate : $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$

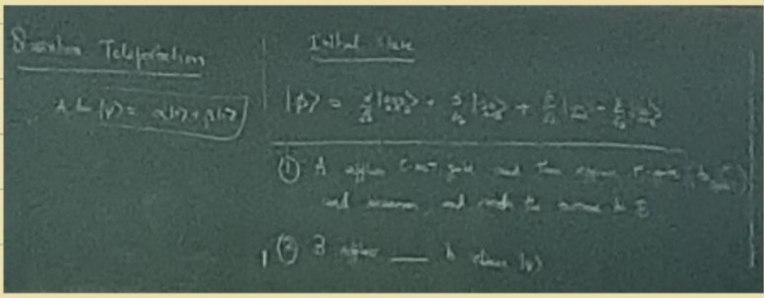
Z - gate : $\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$

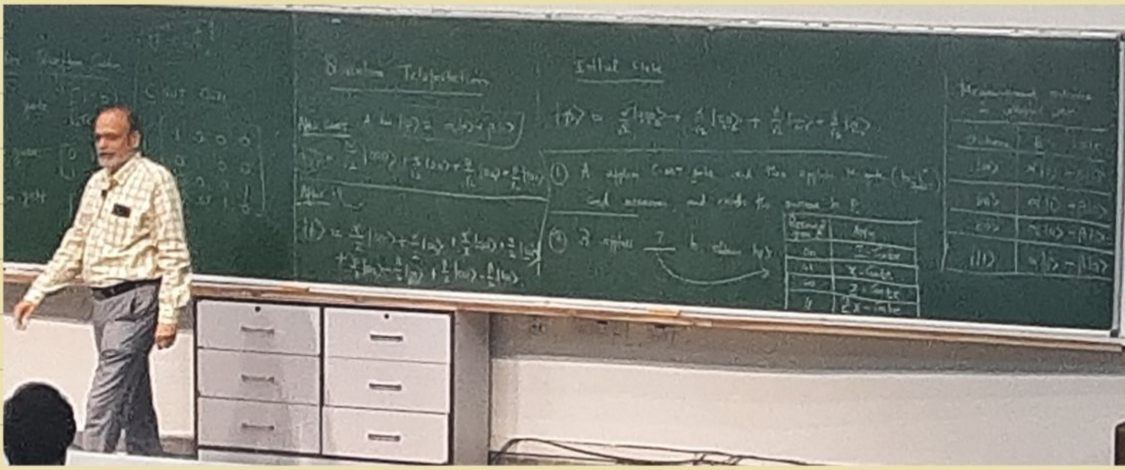
C-NOT gate : $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

$$U^{-1} = U^{T}$$

→ **Quantum teleportation :**





Dense Coding is the opposite of Quantum teleportation.

Next Class : Shor's Algorithm to break RSA using Quantum Algorithms

Problem Set 6

Fermat's little Theorem:

For any prime & a not divisible by p, $\{ gcd(a,p) = 1 \}$

$a^{p-1} \equiv 1 \pmod p$, $a \geq 2$, $a \leq p-1$

If prime $\Rightarrow$ Test always pass

Negligible functions:

$\mu(n)$ is negligible if $\forall$ +ve polynomial, $P(n)$, $\exists N$ S.T. $n > N$, $\mu(n) < \frac{1}{P(n)}$

Inverse functions:

Multiplication $\longleftrightarrow$ Factoring
(Easy) (Hard)

e.g. $f(x) = g^x \cdot \bmod p)$

One-way functions:

(1) Hard to Invert
(2) Length preservation
(3) Exists if $P \neq NP$

e.g. RSA : $f(x) = x^3 \bmod (N)$, $N = pq$

If OWF exists $\Rightarrow \exists$ algorithm to compute it in n^k time.

$\therefore$ Pad the input or modify the machine for n^2.

OWF $\longrightarrow$ Poly-time efficient, i.e. $|n|^k$

Fermat Little Theorem Proof:

$\{1, 2, 3, \ldots\ldots, p-1\}$
$\{a, 2a, 3a, \ldots\ldots, a(p-1)\}$

$$1 \cdot 2 \cdot \ldots, (p-1) \equiv a \cdot 2a \cdot \ldots\ldots \cdot (p-1)a \bmod p$$
$$\Rightarrow (p-1)! \equiv a^{p-1} \cdot (p-1)! \bmod p$$
$$\Rightarrow 1 \equiv a^{p-1} \cdot \bmod p$$

$gcd((p-1)!, p) = 1$

$\rightarrow$ **RSA:**

$p, q \rightarrow$ large primes
$N = pq$
$\varphi(N) = (p-1)(q-1)$
$gcd(e, \varphi(N)) = 1$
$d = e^{-1} \bmod (\varphi(N))$

Public Key : (N, e)
Private Key : (N, d, p, q)

Encryption : $C = M^e \bmod N \leftarrow E(M)$
Decryption : $M = C^e \bmod N$
$M^{ed} = M^{1+k\varphi(N)} \equiv M \bmod N$

Byzantine - Unpredictive, Malicious

All non-faulty processes $(n-f)$ must :

(i) Agree on a value

(ii) That value was proposed by a non-faulty node.

Quantum :

$|\varphi\rangle = \alpha|0\rangle + \beta|1\rangle$

$P(0) = |\alpha|^2$

$P(1) = |\beta|^2$

$$|+\rangle = \frac{|0\rangle + |1\rangle}{2}$$

$\text{CNOT}(|a\rangle|b\rangle) = |a\rangle|a \oplus b\rangle \Rightarrow \text{CNOT}(|0\rangle|\text{Blank}\rangle) = |0\rangle|0\rangle$

No - Cloning Theorem : No Quantum gate can clone arbitrary unknown quantum states.

→ Shor's Algorithm :

Factor Integer N in Polytime → $O(\log^3 N)$, Breaks RSA

$O(n^3)$ vs $O(2^{n^{1/3}})$

→ Flow :

- Edge connectivity :

Menger's Theorem : The size of min-cut b/w two nodes u and v equals the max. flow b/w them (if edge capacities = 1)

Algo : Pick arbitrary node u , Treat u as source.
Run Max - Flow for every other node v as the sink.
The smallest Max - Flow found among all these pairs is the edge connectivity of the graph.

→ P - NP :

$X \leq_p Y \equiv x \in D \iff f(x) \in Y \iff f(x) \notin /$

→ Fibonacii :

$$F_n = \frac{\phi^n - \varphi^n}{\sqrt{5}} \qquad \phi = \frac{1+\sqrt{5}}{2} \quad \& \quad \varphi = \frac{1-\sqrt{5}}{2}$$

$\bar{\phi}$